

CODERHOUSE
MATERIAL DE LECTURA

AI Engineering

Guía de lectura del curso

Contenido

MÓDULO 1 • La interfaz base: conexión y abstracción de LLMs	4
Arquitectura asíncrona con Python 3.12	4
Abstracción de clientes y configuración robusta	10
PRE-ENTREGA Cliente de LLM robusto y asíncrono	12
MÓDULO 2 • Encadenamiento lógico: orquestación con LangChain	15
Arquitectura de cadenas con LCEL y runnables	15
Validación estructurada y resiliencia en cadenas	23
PRE-ENTREGA Pipeline de procesamiento validado	25
MÓDULO 3 • Persistencia de datos y vector DBs	28
Embeddings y similitud	28
Estrategias de Chunking y preprocesamiento de documentos	31
Persistencia local con ChromaDB: operaciones CRUD	33
PRE-ENTREGA Sistema de recuperación semántica local (RAG)	35
MÓDULO 4 • Escalabilidad documental: RAG avanzado y pinecone	38
Pinecone Serverless: persistencia vectorial en la nube	38
Ingesta masiva y gestión de metadatos avanzados	39
Flashcards: Métricas y recuperación híbrida	40
PRE-ENTREGA Sistema RAG escalable en la nube con Pinecone	43
MÓDULO 5 • Razonamiento autónomo: introducción de agentes con LangGraph	46
Arquitectura de grafos: nodos, aristas y estado en LangGraph	46
Tool Calling: Definición y Ejecución de Herramientas con LangChain	54
Ciclos ReAct y Persistencia de Checkpoints en Agentes Autónomos	56
PRE-ENTREGA Agente de razonamiento cíclico con memoria persistente	58
MÓDULO 6 • Sistemas multi-agente: colaboración y especialización	61
Topologías Multi-Agente: Colaboración vs Jerarquía	61
El Patrón Supervisor para Delegación Dinámica	65
Estado Compartido y Comunicación Asíncrona	66
PRE-ENTREGA Orquestador multi-agente especializado	68

MÓDULO 7 • Producción y robustez: observabilidad, costos y despliegue	71
Observabilidad y Trazado: Arize Phoenix y LangSmith	71
Escalabilidad Asíncrona con FastAPI y Redis	74
PRE-ENTREGA API de producción y monitoreo activo	77
MÓDULO 8 • Capstone: entrega final	81
ENTREGA FINAL Sistema Intelligence de grado de producción	82
Material complementario	84

La interfaz base: conexión y abstracción de LLMs

Construir la capa de abstracción base para interactuar con LLMs utilizando Python 3.12, Asyncio y SDKs oficiales.

Arquitectura asíncrona con Python 3.12

Aprende a implementar patrones de concurrencia asíncrona utilizando las últimas APIs de Python 3.12 para optimizar la interacción con LLMs y construir sistemas de IA escalables.

Bienvenido a la primera unidad del programa AI Engineering. Si estás aquí, es porque ya no te conformas con escribir scripts lineales que llaman a un LLM; estás listo para construir sistemas de inteligencia artificial que soporten cientos de usuarios, gestionen múltiples flujos de datos en paralelo y ofrezcan una experiencia de usuario fluida mediante streaming.

En el mundo de la IA, el tiempo no se gasta "calculando" (eso ocurre en los servidores de OpenAI o Anthropic), sino **esperando**. Esperando a que el prompt llegue, esperando a que el modelo procese y esperando a que los tokens regresen por la red. La arquitectura asíncrona es la herramienta de ingeniería que convierte esa espera pasiva en una oportunidad de ejecución.

1. El Porqué del Asyncio en la Ingeniería de IA

Imagina que estás construyendo un agente de IA que debe:

1. Buscar información en una base de datos vectorial (ChromaDB).
2. Consultar una API externa de clima.
3. Generar una respuesta usando GPT-4.

En un modelo **síncrono (bloqueante)**, tu programa se detendría por completo en el paso 1 hasta recibir respuesta, luego en el 2, y finalmente en el 3. Si cada llamada tarda 2 segundos, tu usuario final esperará al menos 6 segundos. Peor aún, mientras tu servidor "espera", no puede atender a ningún otro usuario.

Asyncio resuelve esto permitiendo que un solo proceso gestione múltiples tareas de I/O (entrada/salida) de forma concurrente. No es paralelismo real (como tener varios procesadores trabajando), sino **concurrencia cooperativa**: mientras la "Tarea A" espera datos de la red, cede el control para que la "Tarea B" avance.

Python 3.12: La madurez del modelo

Python 3.12 no solo trae mejoras de rendimiento, sino que refina la API de `asyncio` para que sea más robusta y menos propensa a errores comunes. Se han eliminado comportamientos antiguos y

ambiguos (como métodos automáticos de creación de loops) para obligar a los desarrolladores a seguir patrones más limpios y predecibles.

2. Conceptos Fundamentales: La Anatomía de `asyncio`

A. Corrutinas (`async def` y `await`)

Una corrutina es una función que puede pausar su ejecución. Piénsala como una **receta con marcadores de libros**. Puedes empezar a cocinar (ejecutar código), llegar a un punto donde debes esperar a que el horno se caliente (`await`), poner un marcador ahí, y mientras tanto, empezar a picar verduras para otra receta.

CÓDIGO

```
async def fetch_llm_response(prompt: str):
    # La ejecución se pausa aquí, liberando el hilo para otras tareas
    response = await client.chat.completions.create(model="gpt-4", messages=[...])
    return response
```

B. El Event Loop (El Gestor del Restaurante)

El Event Loop es el motor invisible que coordina las corrutinas. Es como un gestor de restaurante extremadamente eficiente:

- Recibe pedidos (Tareas).
- Los asigna.
- Si un camarero está esperando que la cocina entregue un plato, el gestor le ordena atender otra mesa inmediatamente.

C. Tasks (Tareas)

Una tarea es una corrutina que ha sido programada para ejecutarse "lo antes posible" en el event loop. Mientras que una corrutina por sí sola no hace nada hasta que la esperas (`await`), una `Task` permite que el código se ejecute en segundo plano mientras tú sigues con otras operaciones.

3. Implementación de Patrones de Concurrency en IA

A partir de Python 3.12, la forma de interactuar con el loop ha cambiado. Ya no deberías usar `asyncio.get_event_loop()` de forma descuidada.

El punto de entrada moderno: `asyncio.run()`

Este es el estándar de oro para iniciar tu aplicación. Gestiona la creación del loop, la finalización de las tareas y el cierre limpio de los ejecutores.

CÓDIGO

```
import asyncio

async def main():
    print("Iniciando sistema de IA...")
    # Lógica de la aplicación aquí

if __name__ == "__main__":
    asyncio.run(main())
```

Concurrencia con `asyncio.gather`

Este es el patrón más común cuando necesitas disparar múltiples llamadas a modelos de lenguaje simultáneamente. Por ejemplo, si quieres comparar las respuestas de tres modelos diferentes sobre el mismo prompt:

CÓDIGO

```
async def compare_models(prompt):
    # Programamos las tres llamadas para que ocurran al mismo tiempo
    results = await asyncio.gather(
        call_openai(prompt),
        call_anthropic(prompt),
        call_local_llama(prompt),
        return_exceptions=True # Crucial para que un fallo no tumbe todo
    )
    return results
```

Escenario Real: En una canalización de RAG (Generación Aumentada por Recuperación), podrías usar `gather` para buscar en tu base de datos vectorial y, al mismo tiempo, hacer una búsqueda en Google Search API. No hay razón para esperar a que termine una para empezar la otra.

4. Gestión de Tareas y Latencia en Producción

Como Arquitecto de IA, tu prioridad no es solo que el código funcione, sino que sea resiliente.

Timeouts: El seguro contra la latencia infinita

Los LLMs pueden volverse lentos o las APIs pueden fallar. Nunca debes permitir que una llamada a un LLM quede abierta indefinidamente.

CÓDIGO

```
try:
    async with asyncio.timeout(10.0): # Nuevo en Python 3.11+, estándar en 3.12
        response = await call_llm(prompt)
except TimeoutError:
    print("El modelo tardó demasiado. Aplicando estrategia de fallback...")
```

Semáforos: Control de flujo y Rate Limits

Si intentas disparar 1,000 llamadas a OpenAI en un segundo usando `asyncio.gather`, recibirás un error de `429: Too Many Requests`. Los `asyncio.Semaphore` te permiten limitar cuántas tareas concurrentes pueden estar activas.

CÓDIGO

```
sem = asyncio.Semaphore(5) # Solo 5 llamadas concurrentes permitidas

async def throttled_call(prompt):
    async with sem:
        return await call_llm(prompt)
```

5. Errores Comunes y Malentendidos (Anti-patrones)

"Async hace que el procesamiento de datos sea más rápido"

FALSO. Asyncio no acelera el cálculo matemático. Si estás haciendo limpieza de datos pesada con Pandas o corriendo una inferencia de PyTorch localmente en la CPU, `asyncio` no ayudará. Para tareas **CPU-bound**, necesitas `multiprocessing`. Asyncio es para tareas **I/O-bound** (red, bases de datos, APIs).

El "Bloqueo del Loop"

Este es el error más peligroso. Si dentro de una función `async` llamas a una función síncrona que tarda mucho (como `time.sleep(5)` en lugar de `await asyncio.sleep(5)`), **detendrás a todos tus usuarios**. El gestor del restaurante se queda congelado mirando una mosca, y nadie más recibe su comida.

Solución en Python 3.12: Si debes usar una librería antigua que no es asíncrona, usa `asyncio.to_thread()`:

CÓDIGO

```
# Esto ejecuta la función pesada en un hilo separado sin bloquear el loop principal
result = await asyncio.to_thread(blocking_sync_function, data)
```

6. Aplicaciones en la Industria: El caso del Streaming de Tokens

Una de las aplicaciones más críticas del asincronismo en IA es el **Streaming**. Los usuarios de ChatGPT esperan ver las palabras aparecer una a una. Esto se logra mediante generadores asíncronos.

En un entorno síncrono, el servidor tendría que esperar a que se generen los 500 tokens de una respuesta para enviárselos al cliente. Con `asyncio` y el SDK de OpenAI (que usa `httpx` por debajo de forma asíncrona), podemos procesar cada token según llega de la red y enviarlo al frontend mediante WebSockets o Server-Sent Events (SSE).

Esto reduce el **Time to First Token (TTFT)**, una de las métricas de UX más importantes en productos de IA actuales.

7. Ejemplo resuelto paso a paso: batch de embeddings con control de concurrencia

Hasta acá viste las piezas por separado (`gather` , `Semaphore` , `timeout`). Ahora las juntamos en un caso real y **completamente resuelto**: procesar una lista de textos generando sus embeddings de forma concurrente, pero respetando un límite de llamadas simultáneas, con timeout por llamada y sin que un fallo tumbe todo el lote.

El código corre tal cual (no necesita API keys: simulamos la latencia de red).

CÓDIGO

```
import asyncio
import random
import time

# --- 1. Simulamos una llamada a la API de embeddings (tarea I/O-bound) ---
async def get_embedding(text: str, sem: asyncio.Semaphore) -> dict:
    async with sem:
        latency = random.uniform(0.5, 2.5)      # respeta el límite de concurrencia
        # latencia de red simulada
    try:
        async with asyncio.timeout(2.0):
            await asyncio.sleep(latency)        # await, NO time.sleep (bloquearía
    el loop)

        return {"text": text, "dim": 1536, "ok": True}
    except TimeoutError:
        return {"text": text, "ok": False, "error": "timeout"}

# --- 2. Orquestador: dispara todas las tareas concurrentemente ---
async def embed_batch(texts: list[str]) -> list[dict]:
    sem = asyncio.Semaphore(3)                  # máximo 3 llamadas en paralelo
    tasks = [get_embedding(t, sem) for t in texts]
    return await asyncio.gather(*tasks, return_exceptions=True)

# --- 3. Punto de entrada ---
async def main():
    textos = [f"documento_{i}" for i in range(8)]
    inicio = time.perf_counter()
    resultados = await embed_batch(textos)
    elapsed = time.perf_counter() - inicio

    ok = [r for r in resultados if isinstance(r, dict) and r.get("ok")]
    fallidos = [r for r in resultados if isinstance(r, dict) and not r.get("ok")]

    print(f"Procesados OK: {len(ok)}/{len(textos)} en {elapsed:.2f}s")
    for f in fallidos:
        print(f" Falló: {f['text']} -> {f['error']}")

if __name__ == "__main__":
    asyncio.run(main())
```

Salida esperada (aproximada, varía por la latencia aleatoria):

Procesados OK: 6/8 en 3.42s Falló: documento_2 -> timeout Falló: documento_5 -> timeout

Qué pasó, paso a paso:

1. **asyncio.Semaphore(3)** actúa como un "portero": aunque creamos 8 tareas, solo 3 pueden estar dentro del bloque `async with sem` a la vez. Las otras esperan su turno. Por eso 8 textos se procesan en ~3 "tandas", no todos de golpe.

2. `asyncio.gather(*tasks, return_exceptions=True)` dispara las 8 tareas y espera a todas. El `return_exceptions=True` es clave: si una falla, el lote no se cae; el error vuelve como un resultado más.
3. `asyncio.timeout(2.0)` envuelve cada llamada: si la latencia simulada supera los 2 segundos, se lanza `TimeoutError`, lo capturamos y devolvemos un resultado marcado como fallido en vez de romper el programa.
4. `await asyncio.sleep(latency)` simula la espera de red sin bloquear el event loop. Si hubiéramos usado `time.sleep`, habríamos congelado a todos los usuarios (el anti-patrón de la sección 5).

Conexión con la pregunta de reflexión: este es exactamente el patrón que usarías para embeddear 10.000 filas. Nunca harías `gather` de 10.000 llamadas a la vez (te ganarías un `429: Too Many Requests`): las encolás todas, pero el semáforo asegura que solo N corran en simultáneo.

8. Síntesis y Mejores Prácticas

Para cerrar esta unidad, recuerda estas reglas de oro para tu arquitectura en Python 3.12:

1. Usa `asyncio.run()` como único punto de entrada para tus scripts.
2. Prefiere `asyncio.get_running_loop()` sobre la versión antigua si necesitas acceder al loop desde dentro de una función.
3. Nunca bloques el loop. Identifica funciones síncronas de I/O y envuélvelas en `to_thread()`.
4. Implementa Timeouts y Semáforos desde el día 1. Un sistema de IA sin control de flujo es una bomba de tiempo hacia el agotamiento de cuotas y latencias extremas.
5. Aprovecha `asyncio.TaskGroup` (introducido en 3.11 y refinado en 3.12) para gestionar grupos de tareas de forma más segura que `gather`, asegurando que si una falla, las demás se cancelen limpiamente.

En la siguiente unidad, daremos el paso lógico: utilizar estos cimientos asíncronos para construir **clientes de LLM robustos**, capaces de alternar entre proveedores como OpenAI y Anthropic sin cambiar una sola línea de lógica de negocio.

Pregunta para reflexionar: Si tuvieras que procesar un dataset de 10,000 filas haciendo un embedding de cada una, ¿usarías `asyncio.gather` con las 10,000 llamadas a la vez? ¿Por qué? (Pista: Revisa la sección de Semáforos).

Abstracción de clientes y configuración robusta

Aprende a estructurar la capa de conexión con LLMs utilizando patrones de diseño, validación de tipos con Pydantic y gestión segura de secretos en Python 3.12.

En la arquitectura de sistemas de IA, la **capa de conexión** es el cimiento de todo el flujo. Si esta capa es frágil, tu sistema sufrirá de tiempos de inactividad, fugas de secretos y dificultades para escalar o cambiar de proveedor.

¿Por qué abstraer los clientes?

Muchos desarrolladores cometen el error de instanciar clientes de OpenAI o Anthropic directamente en su lógica de negocio. Esto genera tres problemas graves:

1. **Acoplamiento:** Cambiar de GPT-4o a Claude 3.5 requiere reescribir docenas de líneas de código.
2. **Dificultad de Testing:** Es complejo simular (mockear) llamadas a la API si el cliente está "enterrado" en la lógica.
3. **Inconsistencia:** Cada SDK tiene firmas diferentes (`choices[0].message` vs `content[0].text`).

Patrón Factory y Validación con Pydantic

Para solucionar esto, utilizamos un **Factory Pattern**. Este patrón permite que el sistema decida qué cliente usar en tiempo de ejecución basándose en la configuración.

Además, en Python 3.12, la validación de tipos con **Pydantic** se vuelve obligatoria para manejar secretos de forma segura con `SecretStr`, evitando que tus llaves de API aparezcan accidentalmente en los logs de error.

Diferencias Clave que debes abstraer

- **OpenAI:** Estructura de mensajes basada en `role` y `content`. Respuesta en `choices`.
- **Anthropic:** Estructura similar pero con diferencias en la extracción del texto (`content` es una lista de objetos).

A continuación, implementaremos una base sólida que maneje estas diferencias de forma transparente para el resto de tu aplicación.

Cliente de LLM robusto y asíncrono

Guía detallada para la construcción de una capa de abstracción asíncrona para LLMs empleando estándares de ingeniería de software para IA.

Qué construir

Debes entregar un **repositorio de código** (o un archivo Python estructurado) que contenga la implementación de un **"Unified Async LLM Client"**.

Este artefacto debe ser una clase (o conjunto de clases) en Python 3.12 que permita:

1. **Intercambiabilidad:** Poder instanciar un proveedor (OpenAI o Anthropic) bajo una interfaz común.
2. **Asincronía:** Todas las llamadas a modelos deben ser no bloqueantes (`async/await`).
3. **Streaming:** Implementar un método que devuelva un generador asíncrono de tokens.
4. **Validación:** Uso de Pydantic para definir la estructura de los mensajes de entrada y la configuración del modelo.

Pasos Sugeridos

1. **Estructura de Datos:** Crea un archivo `schemas.py` con Pydantic para definir `ChatMessage` (role, content) y `ModelResponse` . Implementar esto primero evita el "error de diccionarios anidados" tan común en principiantes.
2. **La Base Asíncrona:** Define una clase base abstracta `BaseLLMClient` con un método `async def generate()` .
3. **Implementación del Proveedor:** Crea `OpenAIClient` y `AnthropicClient` heredando de la base. Usa los SDKs oficiales (`openai` y `anthropic`) en sus versiones asíncronas (`AsyncOpenAI` y `AsyncAnthropic`).
4. **Lógica de Streaming:** Implementa el generador usando `yield` dentro de un loop `async for` que recorra el stream del SDK.
5. **script de validación:** Crea un archivo `main.py` simple que importe tus clientes, cargue el `.env` y realice una pregunta corta ("¿Qué es la entropía?") tanto en modo normal como en streaming.

Errores Comunes a Evitar

- **Bloqueo del Event Loop:** Un error clásico de nivel intermedio es usar la versión síncrona del cliente dentro de una función `async` . Esto detiene todo el programa mientras el LLM "piensa". Asegúrate de usar `await client.chat.completions.create(...)` .
- **Fuga de Excepciones:** No dejes que un error de "API Key inválida" o "Límite de cuota" rompa el loop principal. Captura la excepción y devuelve un mensaje estructurado o haz un reintento (retry).

Qué entregás y en qué formato

- **Tipo:** Código — un repositorio de GitHub (no es un documento ni un PDF).
- **Artefacto concreto:** repo con los clientes async (OpenAI + Anthropic), `schemas.py` (Pydantic), `.env.example`, un `main.py` que pruebe el modo normal y el streaming, y un `README.md`.
- **Qué NO hace falta:** no entregás informe ni documento aparte; toda la explicación (cómo correrlo, variables de entorno) va en el `README.md`.

Pasos para la entrega

1. Configura un entorno virtual con Python 3.12 e instala `openai`, `anthropic`, `pydantic` y `python-dotenv`.
2. Crea un esquema de Pydantic para validar los parámetros de entrada del modelo (temperatura de 0 a 2, `max_tokens`, etc.).
3. Implementa una clase `AsyncLLMManager` que pueda cargar tanto OpenAI como Anthropic basándose en una variable de configuración.
4. El método de generación debe ser capaz de manejar streaming: usa `yield` para retornar fragmentos de texto conforme lleguen de la API.
5. Asegúrate de capturar excepciones de red y de límite de tasa (rate limiting) devolviendo un error controlado en lugar de un crash.
6. Incluye un archivo `README.md` que explique cómo ejecutar el script de prueba y qué variables de entorno son necesarias.

Entregable

Formato: repositorio

Repositorio de GitHub que contenga la implementación de los clientes asíncronos, archivos de configuración (`schemas.py`), variables de entorno (`.env.example`) y un script de prueba de streaming.

Objetivos de aprendizaje

- Construir una clase de cliente asíncrono utilizando los SDKs oficiales de OpenAI y Anthropic.
- Implementar validación de configuraciones y esquemas de mensajes mediante Pydantic.
- Gestionar el streaming de tokens mediante generadores asíncronos en Python 3.12.
- Aplicar patrones de manejo de errores para mejorar la resiliencia de las llamadas a modelos externos.

Rúbrica de evaluación

Esta rúbrica evalúa la capacidad del estudiante para construir una capa de abstracción técnica sólida, utilizando programación asíncrona avanzada y validación de esquemas para interactuar con proveedores líderes de LLM en el marco del programa AI Architect 2026.

Criterio	Descripción	Peso
Abstracción y Gestión de Proveedores (AsyncLLMManager)	Evalúa la implementación de la clase principal para manejar dinámicamente OpenAI y Anthropic mediante configuración.	35%
Implementación de Streaming Asíncrono	Manejo de generadores asíncronos para el procesamiento de tokens en tiempo real.	25%
Validación de Esquemas con Pydantic	Uso de Pydantic para garantizar la integridad y el rango de los parámetros de entrada.	20%
Resiliencia y Manejo de Errores	Capacidad del sistema para manejar fallos de red y Rate Limiting sin interrumpir el flujo del programa.	20%

Total: 100 pts | Aprobación: 70 pts

Encadenamiento lógico: orquestación con LangChain

Diseñar y ejecutar cadenas lógicas de procesamiento utilizando LangChain y LCEL para flujos de trabajo predecibles.

Arquitectura de cadenas con LCEL y runnables

Aprende a diseñar pipelines de IA profesionales utilizando LangChain Expression Language (LCEL). Esta unidad enseña a componer flujos de trabajo escalables, asíncronos y robustos mediante el protocolo Runnable.

Arquitectura de Cadenas con LCEL y Runnables

Bienvenido a la primera unidad del módulo de **Encadenamiento Lógico**. Hasta ahora, en el módulo anterior, has aprendido a conectar con LLMs como OpenAI y Anthropic utilizando sus SDKs base y gestionando la asincronía con Python 3.12. Has visto cómo fluye el texto y cómo controlar la latencia. Sin embargo, en el desarrollo de software profesional, las aplicaciones de IA rara vez consisten en una única llamada a un modelo.

Imagina que estás construyendo un sistema de análisis de sentimientos que primero debe traducir el texto, luego clasificarlo, extraer entidades clave en formato JSON y finalmente generar un resumen ejecutivo. Si usaras código imperativo tradicional (muchos `if`, `await` y transformaciones manuales), navegarías en un "código espagueti" difícil de testear y escalar.

Aquí es donde entra **LangChain Expression Language (LCEL)**. En esta unidad, aprenderás a transformar esos procesos aislados en tuberías (pipelines) declarativas, robustas y preparadas para producción.

1. El Porqué de LCEL: De Scripts a Tuberías de Ingeniería

El problema de la orquestación manual

Cuando trabajas con SDKs básicos (como el de OpenAI), cada paso de tu lógica requiere manejo manual:

1. **Formateo de variables:** Insertar datos del usuario en un string.
2. **Llamada al modelo:** Gestionar reintentos y sesiones.
3. **Parsing:** Convertir el string de respuesta en algo útil (un objeto Python o un JSON).

Si quieres añadir soporte para **streaming**, **ejecución en paralelo** o **rastreo de errores (tracing)**, el código se vuelve exponencialmente complejo.

La solución: LCEL (LangChain Expression Language)

LCEL es un lenguaje declarativo diseñado para componer cadenas de manera intuitiva. Se basa en una idea fundamental: **cada componente es un "bloque" con una interfaz estándar**.

La ventaja principal de LCEL no es solo la sintaxis elegante (el uso del operador pipe `|`), sino lo que obtienes "gratis" al usarlo:

- **Soporte de primera clase para Async:** Puedes alternar entre síncrono y asíncrono sin cambiar la lógica.
- **Streaming automático:** Si el modelo soporta streaming, la cadena lo expone inmediatamente.
- **Ejecución en paralelo:** Si tienes dos tareas independientes (ej. resumir y extraer keywords), LCEL puede ejecutarlas en paralelo sin configurar `asyncio.gather` manualmente.
- **Observabilidad:** Se integra nativamente con herramientas como LangSmith para ver exactamente qué pasó en cada eslabón de la cadena.

2. El Protocolo Runnable: La Interfaz Universal

Para que LCEL funcione, todos los componentes deben hablar el mismo idioma. En LangChain, ese idioma es el **Protocolo Runnable**.

Casi todos los objetos en LangChain (modelos, prompts, parsers, herramientas) heredan de la clase base `Runnable`. Esto garantiza que todos implementen los mismos métodos estratégicos:

Método	Descripción
<code>invoke()</code>	Llama al componente con una entrada única (síncrono).
<code>ainvoke()</code>	Versión asíncrona de <code>invoke</code> .
<code>batch()</code>	Llama al componente con una lista de entradas (optimiza latencia).
<code>stream()</code>	Devuelve fragmentos de la respuesta a medida que se generan.

Analogía de la Estación de Trabajo: Imagina una fábrica de coches. Cada estación (Runnable) tiene una entrada de piezas y una salida de componentes más complejos. No importa si la estación pone puertas o pinta el chasis; todas aceptan piezas en una cinta transportadora y entregan el resultado a la siguiente. El operador `|` es la cinta transportadora que conecta estas estaciones.

3. Composición de Cadenas: El Operador Pipe |

La sintaxis core de LCEL utiliza el operador `|` (pipe), similar a como se usa en terminales de Unix o en lenguajes funcionales. La salida del componente a la izquierda se convierte en la entrada del componente a la derecha.

Estructura típica de una cadena

Un flujo estándar de IA suele verse así: `PromptTemplate | ChatModel | OutputParser`

1. **PromptTemplate (Input -> PromptValue):** Toma un diccionario de variables y genera el mensaje formateado.

2. **ChatModel (PromptValue -> Message)**: Toma el mensaje y lo envía al LLM.
3. **OutputParser (Message -> String/JSON)**: Toma la respuesta cruda del modelo y la limpia.

Ejemplo concreto:

Si queremos un traductor inteligente:

CÓDIGO

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# 1. Definimos el Prompt
prompt = ChatPromptTemplate.from_template("Translate the following text to
{language}: {text}")

# 2. El Modelo
model = ChatOpenAI(model="gpt-4o")

# 3. El Parser
parser = StrOutputParser()

# 4. Composición de la cadena
chain = prompt | model | parser

# 5. Ejecución
# Observa que pasamos un diccionario que matchea las variables del prompt
response = await chain.ainvoke({"language": "Spanish", "text": "Learning LCEL is
powerful."})
print(response) # "Aprender LCEL es poderoso."
```

¿Qué está pasando bajo el capó?

Cuando ejecutas `chain.ainvoke(...)`:

1. El diccionario fluye al `prompt`. El `prompt` genera un objeto `ChatPromptValue`.
2. Ese objeto pasa al `model`. El `model` entiende ese tipo de entrada y genera un `BaseMessage`.
3. El `BaseMessage` pasa al `parser`, que extrae el contenido de texto (el `.content`) y devuelve un string limpio.

4. Gestión Avanzada de Prompts en LCEL

En sistemas escalables, los prompts no son simples strings; son **artefactos de software**. Deben ser modulares, versionables y capaces de manejar estructuras complejas.

ChatPromptTemplate: Más que solo texto

A diferencia de los prompts básicos, `ChatPromptTemplate` permite definir roles (System, Human, AI), lo cual es crucial para modelos modernos que dependen del contexto del sistema para seguir instrucciones de seguridad o formato.

CÓDIGO

```
template = ChatPromptTemplate.from_messages([
    ("system", "Eres un arquitecto de software experto en {topic}."),
    ("human", "Explícame los beneficios de {subtopic} en 3 puntos.")
])
```

Modelado de Entradas (Input Shaping)

Un error común para los desarrolladores es intentar "limpiar" los datos antes de enviarlos a la cadena. LCEL nos permite hacerlo *dentro* de la cadena usando `RunnablePassthrough` o funciones lambda.

Si tu cadena espera un diccionario con `context` y `question`, pero tu base de datos solo te da la `question`, puedes modelar la entrada así:

CÓDIGO

```
from langchain_core.runnables import RunnablePassthrough

# Esta cadena 'inyecta' datos antes de llegar al prompt
map_chain = (
    {"context": lambda x: "Información secreta sobre " + x["topic"], "question":
    lambda x: x["question"]}
    | prompt
    | model
)
```

5. Operaciones por Lote y Paralelismo

Una de las joyas de LCEL es la capacidad de ejecutar múltiples tareas simultáneamente sin complejidad adicional.

Ejecución en Paralelo (`RunnableParallel`)

Imagina que quieres evaluar una idea de negocio desde dos perspectivas: técnica y financiera.

CÓDIGO

```
from langchain_core.runnables import RunnableParallel

chain_tecnica = prompt_tecnico | model | parser
chain_financiera = prompt_financiero | model | parser

# Ejecuta ambas al mismo tiempo
analisis_total = RunnableParallel(tech=chain_tecnica, finance=chain_financiera)

resultado = await analisis_total.ainvoke({"idea": "Drones que entregan pizza"})
# resultado será: {"tech": "...", "finance": "..."}

```

Esto reduce el tiempo total de respuesta al máximo posible (el tiempo de la rama más lenta), en lugar de sumar ambos tiempos.

Procesamiento por Lote (batch)

Si tienes 100 reseñas de clientes para analizar, usar un bucle `for` con `ainvoke` es ineficiente. El método `abatch` optimiza internamente las llamadas para ejecutarlas de la manera más rápida que permita el proveedor.

CÓDIGO

```
inputs = [
    {"language": "French", "text": "Hello"},
    {"language": "German", "text": "Goodbye"},
]
results = await chain.abatch(inputs)
```

6. Errores Comunes y Mejores Prácticas

Error 1: Mismatch de tipos de datos

El error: Intentar pasar un string a una cadena que espera un diccionario. **La solución:** Si tu `ChatPromptTemplate` tiene variables (ej. `{text}`), la entrada de la cadena **debe** ser un diccionario `{"text": "valor"}`. Si el prompt no tiene variables, puedes pasar el string directamente.

Error 2: Ignorar el Output Parser

El error: No usar un parser y recibir un objeto `AIMessage` en lugar de texto. **La solución:** Siempre termina tus cadenas de texto con `StrOutputParser()`. Esto asegura que el resto de tu aplicación reciba un string estándar de Python.

Mejor Práctica: Usar `ainvoke` por defecto

En aplicaciones web (como FastAPI), siempre prefiere los métodos asíncronos (`ainvoke`, `abatch`, `astream`). Esto evita que el loop de eventos de Python se bloquee mientras esperas la respuesta de la API de OpenAI, permitiendo que tu servidor atienda a otros usuarios.

7. Ejemplo resuelto paso a paso: pipeline de análisis de reseñas (end-to-end)

Hasta acá viste cada pieza por separado (pipe, `RunnableParallel`, parser). Ahora las combinamos en un caso real y resuelto: una cadena que, a partir de una reseña de cliente, **en paralelo** clasifica el sentimiento y extrae el tema, y luego **sintetiza** todo en una línea de ticket para el equipo de soporte.

CÓDIGO

```
import asyncio
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableParallel

model = ChatOpenAI(model="gpt-4o", temperature=0)
parser = StrOutputParser()

# --- Sub-cadena 1: sentimiento ---
sentiment_chain = (
    ChatPromptTemplate.from_template(
        "Clasificá el sentimiento de esta reseña en una palabra "
        "(positivo/negativo/neutro): {review}"
    )
    | model
    | parser
)

# --- Sub-cadena 2: tema principal ---
topic_chain = (
    ChatPromptTemplate.from_template(
        "¿Cuál es el tema principal de esta reseña? Respondé en 3 palabras: {review}"
    )
    | model
    | parser
)

# --- Paso 1: corremos ambas sub-cadenas EN PARALELO ---
analysis = RunnableParallel(sentiment=sentiment_chain, topic=topic_chain)

# --- Paso 2: síntesis en una línea de ticket ---
ticket_prompt = ChatPromptTemplate.from_template(
    "Sentimiento: {sentiment}\nTema: {topic}\n"
    "Escribí una sola línea de ticket de soporte (máx. 15 palabras) para el equipo."
)

# --- Cadena completa: análisis paralelo -> síntesis ---
full_chain = analysis | ticket_prompt | model | parser

async def main():
    resultado = await full_chain.ainvoke(
        {"review": "Llevo 3 días esperando que respondan mi reclamo y nadie me contesta."}
    )
    print(resultado)

if __name__ == "__main__":
    asyncio.run(main())
```

Salida esperada (ejemplo):

CÓDIGO

```
Cliente sin respuesta a su reclamo hace 3 días – sentimiento negativo, priorizar.
```

Qué pasó, paso a paso:

1. `ainvoke({"review": ...})` entrega el diccionario a `RunnableParallel`, que se lo pasa **a las dos sub-cadenas a la vez**. Ambas usan la variable `{review}`. No esperamos a que termine una para empezar la otra: el tiempo total es el de la rama más lenta, no la suma.
2. Cada sub-cadena es un `prompt | model | parser` completo, así que devuelve **texto limpio** (gracias a `StrOutputParser`), no un objeto `AIMessage`.
3. `RunnableParallel` arma un diccionario `{"sentiment": "...", "topic": "..."}.` Ese diccionario **fluye directo** a `ticket_prompt`, cuyas variables `{sentiment}` y `{topic}` coinciden con esas claves (acá está la clave de por qué los nombres deben matchear).
4. El paso final vuelve a pasar por `model | parser` para producir la línea de ticket como string listo para usar.

Este patrón —descomponer en sub-cadenas, paralelizar lo independiente y sintetizar al final— es la base de cualquier pipeline LCEL de producción.

Segundo ejemplo resuelto: cadena secuencial + procesamiento por lote

El primer ejemplo mostró ramas en paralelo. Este muestra el patrón opuesto y igual de común: **encadenar pasos en secuencia** (la salida de uno es la entrada del siguiente) y procesar **varios inputs a la vez** con `abatch`. Caso: generar una descripción de producto y luego condensarla a un tweet.

CÓDIGO

```
import asyncio
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

model = ChatOpenAI(model="gpt-4o", temperature=0.7)
parser = StrOutputParser()

# --- Paso 1: generar una descripción de producto ---
descripcion_chain = (
    ChatPromptTemplate.from_template(
        "Escribí una descripción de producto de 2 oraciones para: {producto}"
    )
    | model
    | parser
)

# --- Paso 2: condensar esa descripción a un tweet ---
tweet_chain = (
    ChatPromptTemplate.from_template(
        "Resumí esto en un tweet de menos de 120 caracteres, con 1 emoji: {descripcion}"
    )
    | model
    | parser
)

# --- Cadena secuencial: la salida (string) del paso 1 se "envuelve" en el dict
# que espera el paso 2. La función lambda se convierte en Runnable
# automáticamente. ---
full_chain = descripcion_chain | (lambda desc: {"descripcion": desc}) | tweet_chain

async def main():
    # .abatch procesa varios productos a la vez (más eficiente que un for con
    # ainvoke)
    productos = [{"producto": "termo de acero 1L"}, {"producto": "mochila
    antirrobo"}]
    tweets = await full_chain.abatch(productos)
    for p, t in zip(productos, tweets):
        print(f"{p['producto']}: {t}")

if __name__ == "__main__":
    asyncio.run(main())
```

Salida esperada (ejemplo):

termo de acero 1L: Frío 24h o calor 12h, donde vayas. Tu bebida siempre lista mochila antirrobo: Tu laptop y tus cosas, siempre seguras. Salí tranquilo

Qué pasó, paso a paso:

1. `descripcion_chain` produce un **string** (gracias a `StrOutputParser`).
2. Ese string no se puede pasar tal cual a `tweet_chain`, porque su prompt espera la variable `{descripcion}`. Por eso intercalamos `(lambda desc: {"descripcion": desc})`: LCEL convierte cualquier función en un `Runnable`, así que **modelás la entrada** sin código extra.
3. `tweet_chain` recibe ese diccionario y genera el tweet final.
4. `abatch([...])` ejecuta toda la cadena para **los dos productos a la vez**, optimizando las llamadas. Para 200 productos, esto es muchísimo más rápido que un `for`.

Mientras el primer ejemplo paralelizaba tareas independientes, este encadena pasos dependientes: son las dos formas básicas de componer en LCEL.

8. Síntesis y Conexiones

LCEL no es magia; es una abstracción poderosa sobre el protocolo **Runnable**. Al usarlo, dejas de escribir "código de pegamento" y empiezas a diseñar **arquitectura de datos para IA**.

Puntos clave para recordar:

1. **Composabilidad:** Usa `|` para unir Prompt -> Modelo -> Parser.
2. **Interfaz Única:** Todos los componentes implementan `invoke`, `batch` y `stream`.
3. **Declarativo:** El código describe el *qué* (el flujo), no el *cómo* (la gestión de hilos/estados).

En la siguiente unidad, daremos un paso más allá aprendiendo cómo forzar al modelo a responder con esquemas de datos estrictos (JSON) y cómo manejar errores cuando el flujo se rompe. Por ahora, es momento de que refactorices tu conocimiento previo hacia este nuevo paradigma.

Validación estructurada y resiliencia en cadenas

Aprende a garantizar la fiabilidad de tus cadenas de LangChain mediante la validación estricta de esquemas con Pydantic y la implementación de estrategias de reintento para fallos transitorios.

En la ingeniería de software tradicional, los contratos de datos (como una especificación OpenAPI o una interfaz de TypeScript) aseguran que dos sistemas se comuniquen sin errores de formato. En el mundo de la IA, este principio es aún más crítico: los LLMs son probabilísticos y su salida natural es texto libre, lo cual es veneno para la escalabilidad.

Esta unidad se centra en transformar la "magia" de las respuestas de texto en **sistemas predecibles y resilientes** mediante dos pilares: la validación estricta de esquemas y el manejo proactivo de errores.

1. El Contrato de Datos: Salidas Estructuradas

Cuando un LLM responde en lenguaje natural, corres el riesgo de que cambie el formato de la respuesta, alucine con claves JSON o devuelva tipos de datos incorrectos (como un string donde esperabas un número).

¿Por qué Pydantic?

Pydantic es la librería de validación de datos estándar en Python moderno. Al integrarla con LangChain, definimos un **esquema formal** que el modelo está obligado a seguir.

- **Validación Sintáctica:** El JSON es íntegro y parseable.
- **Validación Estructural:** Todos los campos requeridos están presentes.
- **Validación Semántica:** Los valores cumplen reglas de negocio (ej. un valor de `confianza` debe estar entre 0 y 1).

El método `with_structured_output`

LangChain abstrae la complejidad de parsear manualmente las salidas mediante este método. Por debajo, utiliza técnicas como el **Tool Calling** o **JSON Mode** de los proveedores (OpenAI, Anthropic) para garantizar que el modelo "encaje" su pensamiento en tu molde.

2. Resiliencia: Manejo de Errores y Reintentos

Un sistema de IA en producción no solo debe ser correcto, debe ser **robusto**. Las llamadas a modelos pueden fallar por:

1. **Errores Transitorios:** Caídas de red, latencia o límites de tasa (429 Rate Limit).
2. **Errores de Formato:** El modelo no pudo seguir exactamente el esquema Pydantic.

Estrategias de Recuperación

En lugar de dejar que el sistema falle y devuelva un error al usuario, implementamos:

- **Retries con Backoff Exponencial:** Si falla, esperamos un poco más antes de intentar de nuevo. Esto evita saturar el servicio y permite que fallos momentáneos se resuelvan solos.
- **Fallbacks:** Si un modelo (ej. GPT-4o) falla repetidamente, podemos redirigir la carga a un modelo más ligero o a una respuesta predefinida.

Errores Comunes de Arquitectura

1. **Confiar ciegamente en el JSON del modelo:** Nunca asumas que un `json.loads` funcionará. Usa siempre un validador de esquema.
2. **Reintentar errores permanentes:** Si el prompt tiene un error lógico que causa rechazo, reintentar 10 veces solo desperdicia dinero y latencia. Diferencia entre errores de transporte (red) y errores de lógica.

En el siguiente ejercicio, aplicarás estos conceptos para convertir una cadena frágil en un pipeline de grado producción.

Pipeline de procesamiento validado

Briefing para la pre-entrega del Módulo 2: Construcción de un pipeline de procesamiento de texto utilizando LangChain (LCEL) y validación estructurada con Pydantic.

Qué construir

Debes desarrollar un **Pipeline de Extracción de Entidades Técnicas**. El sistema debe recibir un párrafo de texto sin procesar (por ejemplo, una descripción de arquitectura de software o un log de error) y devolver un objeto validado.

Los componentes requeridos:

1. **Esquema Pydantic:** Un modelo que defina campos como `tecnologias` (lista de strings), `nivel_de_criticidad` (enum: baja, media, alta), y `resumen_tecnico` (string).
2. **Prompt Template:** Un template modular que acepte el texto de entrada y las instrucciones de formato.
3. **Cadena LCEL:** Una composición que una el Prompt + LLM + Output Parser.
4. **Lógica de Resiliencia:** Configuración de al menos un reintento automático si el LLM devuelve un JSON mal formado o incompleto.

Pasos sugeridos

1. **Define tu Contrato:** Crea la clase Pydantic. Piensa en qué restricciones quieres poner (ej. que la lista de tecnologías no esté vacía).
2. **Prepara el Parser:** Utiliza `PydanticOutputParser` o el método `.with_structured_output()` de LangChain (preferido para OpenAI/Anthropic).
3. **Ensambla la Cadena:**

CÓDIGO

```
chain = prompt | model.with_structured_output(TuEsquema)
```

1. **Añade Resiliencia:** Envuelve la llamada con una estrategia de reintento (`.with_retry()`).
2. **Prueba de Estrés:** Pasa un texto ambiguo y verifica si el validador lanza excepciones o si el modelo se recupera.

Errores comunes a evitar

- **Ignorar el `finish_reason`** : A veces el LLM corta la respuesta por falta de tokens. Tu pipeline debe detectar si el objeto está incompleto antes de intentar transformarlo.

- **Hardcoding de Prompts:** Evita las F-strings de Python dentro de la cadena. Usa `ChatPromptTemplate` para mantener la modularidad y permitir que LangChain gestione las variables de entrada.

Qué entregás y en qué formato

- **Tipo:** Código — un repositorio de GitHub.
- **Artefacto concreto:** repo con `schemas.py` (modelo Pydantic) y `chain.py` (cadena LCEL con `.with_structured_output()` y reintento `.with_retry()`).
- **Qué NO hace falta:** no hay PDF ni informe; el ejemplo de salida JSON y las instrucciones van en el `README.md`.

Pasos para la entrega

1. Crea un archivo `schemas.py` donde definas la estructura de salida deseada usando Pydantic.
2. En `chain.py`, configura un cliente de ChatOpenAI o ChatAnthropic (reutilizando la lógica del Módulo 1).
3. Crea un `ChatPromptTemplate` que instruya al modelo para extraer información técnica de un texto.
4. Construye la cadena usando LCEL: `prompt | model.with_structured_output(Schema)`.
5. Implementa una función asíncrona `process_text(text: str)` que ejecute la cadena usando `.ainvoke()`.
6. Asegúrate de incluir logs adecuados para observar el proceso de validación y posibles reintentos.

Ejemplo de la salida esperada

Dado un texto de entrada (ej. un log de error o descripción de arquitectura), tu pipeline debe devolver un objeto validado como este:

CÓDIGO

```
{
  "tecnologias": ["FastAPI", "Redis", "PostgreSQL"],
  "nivel_de_criticidad": "alta",
  "resumen_tecnico": "API con caché en Redis y persistencia en PostgreSQL; cuello de botella en conexiones concurrentes."
}
```

Checklist de entrega

- Repositorio de GitHub con `schemas.py` (modelo Pydantic) y `chain.py` (cadena LCEL).
- Cadena compuesta con LCEL: `prompt | model.with_structured_output(Schema)`.
- Lógica de reintento (`.with_retry()`) ante JSON mal formado o incompleto.
- Función asíncrona `process_text()` con `.ainvoke()` y logs de validación.
- Mini-script de prueba que ejecute un ejemplo.

Entregable

Formato: repositorio

Repositorio de GitHub con el código del pipeline, incluyendo el modelo Pydantic, el prompt template y la cadena LCEL con lógica de reintentos. Debe incluir un mini-script de prueba asíncrono.

Objetivos de aprendizaje

- Componer cadenas lógicas de procesamiento utilizando LangChain Expression Language (LCEL).
- Implementar contratos de datos estrictos mediante modelos de Pydantic para salidas de LLM.
- Configurar estrategias de reintento y manejo de errores en flujos asíncronos de IA.
- Diseñar prompts modulares que se integren en arquitecturas de software escalables.

Rúbrica de evaluación

Esta rúbrica evalúa la capacidad del estudiante para orquestar flujos de trabajo de IA utilizando LangChain Expression Language (LCEL), validación de datos con Pydantic y ejecución asíncrona, asegurando la escalabilidad y robustez del sistema.

Criterio	Descripción	Peso
Definición del Esquema y Salida Estructurada	Evaluación del uso de Pydantic para definir contratos de datos y la implementación de <code>with_structured_output</code> .	30%
Implementación de Cadena LCEL y Prompting	Uso de la sintaxis de pipe (<code> </code>) de LangChain y diseño de <code>ChatPromptTemplate</code> para la extracción técnica.	30%
Ejecución Asíncrona y Resiliencia	Uso de <code>.ainvoke()</code> y manejo de logs para observar el flujo y la validación.	25%
Estructura del Repositorio y Script de Prueba	Organización del código en módulos (<code>schemas.py</code> , <code>chain.py</code>) y presencia de un script de prueba.	15%

Total: 100 pts | Aprobación: 70 pts

Persistencia de datos y vector DBs

Implementar un sistema de memoria de largo plazo mediante arquitecturas RAG básicas y bases de datos vectoriales locales.

Embeddings y similitud

Explora cómo la IA convierte el lenguaje humano en representaciones vectoriales (embeddings) y cómo las métricas de similitud permiten a las máquinas entender la relevancia semántica para construir sistemas de recuperación de información escalables.

Imagina que intentas buscar un documento sobre "cómo limitar el tráfico de red" en una base de datos tradicional de palabras clave. Si buscas exactamente "limitar tráfico", pero tu documento usa los términos "restricción de ancho de banda" o "rate-limiting", es muy probable que el sistema no lo encuentre. Este es el talón de Aquiles de la búsqueda por palabras clave: la rigidez terminológica.

En esta unidad, aprenderás cómo la Inteligencia Artificial resuelve este problema transformando el lenguaje humano en **geometría**. Al convertir palabras y conceptos en vectores numéricos (embeddings), podemos medir qué tan "cerca" están dos ideas en un espacio multidimensional, permitiendo que nuestras aplicaciones entiendan el significado (semántica) más allá de las letras exactas.

1. ¿Qué es un Embedding?

Un **embedding** es una representación numérica densa de un objeto (usualmente texto, pero también puede ser imagen o audio) en un espacio vectorial de alta dimensionalidad.

En términos más sencillos: es una lista de números (un vector) que captura el "significado" de un fragmento de texto.

El paso de lo discreto a lo denso

Antes de los embeddings modernos, usábamos representaciones "One-Hot Encoding", donde cada palabra era una posición en un vector gigante lleno de ceros. Si tenías un vocabulario de 50,000 palabras, la palabra "perro" era un vector de 50,000 elementos con un solo "1".

- **Problema:** En este sistema, la distancia matemática entre "perro" y "canino" era la misma que entre "perro" y "microprocesador". No había noción de significado.

Los **embeddings** son vectores "densos" (usualmente de entre 1536 y 3072 dimensiones en modelos modernos como los de OpenAI). En este espacio, cada dimensión representa una característica abstracta aprendida por el modelo durante su entrenamiento.

La analogía del Mapa de Significados

Imagina un mapa 2D. Si colocamos "Madrid" en las coordenadas (40, -3) y "Lisboa" en (38, -9), la distancia física entre los puntos refleja su cercanía geográfica. Un embedding hace exactamente lo mismo pero en miles de dimensiones: coloca "Rey" cerca de "Hombre", y "Reina" cerca de "Mujer", manteniendo distancias y relaciones lógicas entre conceptos.

2. La Geometría de la Similitud

A. Similitud Coseno (La más utilizada)

Mide el coseno del ángulo entre dos vectores.

- **Qué captura:** La dirección. No importa si un texto es más largo que otro (vector más grande), lo que importa es que "apunten" hacia el mismo concepto.
- **Rango:** De -1 a 1 (en embeddings de texto suele ser de 0 a 1, donde 1 es identidad total).
- **Caso de uso:** Es el estándar en RAG (Retrieval-Augmented Generation) porque ignora la longitud del texto y se enfoca en el contenido temático.

B. Distancia Euclídea (L2)

Mide la distancia en línea recta entre dos puntos.

- **Problema:** Es muy sensible a la magnitud. Si un vector representa un párrafo y otro una oración, la distancia euclídea puede ser grande aunque traten del mismo tema.
- **Uso:** Común cuando los vectores han sido normalizados previamente.

C. Producto Punto (Dot Product)

Multiplica las magnitudes y el coseno del ángulo.

- **Uso:** Muy común en sistemas de recomendación donde la intensidad o "importancia" de un vector (su magnitud) es relevante.

Nota

Regla de oro de ingeniería: La mayoría de las bases de datos vectoriales (como ChromaDB o Pinecone) te permiten elegir la métrica. Si estás trabajando con modelos de OpenAI o HuggingFace, la **Similitud Coseno** es casi siempre la elección correcta si no has normalizado los vectores.

3. Del Contexto a la Representación

No todos los embeddings son iguales. La evolución tecnológica ha cambiado cómo extraemos este "significado":

1. **Modelos Estáticos (Word2Vec, GloVe):** Cada palabra tenía un único vector. La palabra "banco" (de sentarse) y "banco" (entidad financiera) tenían el mismo embedding. El sistema era "ciego" al contexto.

2. **Modelos Contextuales (BERT, Ada, Voyage):** La misma palabra genera vectores distintos según las palabras que la rodean. El modelo "entiende" que en "me senté en el banco", la palabra se refiere a mobiliario urbano.

El rol de los modelos modernos (OpenAI, Voyage, Cohere)

Como desarrollador, interactuarás con modelos de embeddings a través de APIs. Por ejemplo, el modelo `text-embedding-3-small` de OpenAI toma un string y te devuelve una lista de 1536 flotantes.

Ejemplo conceptual en Python:

CÓDIGO

```
import openai

# Texto A: "El sistema de archivos está lleno"
# Texto B: "No queda espacio en el disco duro"
# Texto C: "Me gusta comer manzanas"

# El modelo generará vectores tales que:
# sim_coseno(Vector A, Vector B) ≈ 0.85 (Alta similitud semántica)
# sim_coseno(Vector A, Vector C) ≈ 0.15 (Baja similitud semántica)
```

4. Aplicaciones en el Mundo Real

Búsqueda Semántica (Semantic Search)

Empresas como **Notion** o **GitHub** utilizan embeddings para sus buscadores. Si un usuario busca "problemas de pagos", el sistema puede recuperar documentos que mencionen "error en la pasarela de Stripe" o "transacción declinada", aunque no compartan ninguna palabra clave.

Deduplicación Diferida

En sistemas de soporte técnico (como **Zendesk**), se pueden agrupar tickets que preguntan lo mismo ("¿cómo cambio mi clave?" vs "¿procedimiento para resetear password?") comparando sus embeddings y evitando que los agentes respondan lo mismo varias veces.

Clasificación de Zero-Shot

Puedes clasificar texto sin entrenar un modelo. Si tienes categorías como "URGENTE", "SOLICITUD" y "SPAM", simplemente calculas el embedding de cada etiqueta y el del correo entrante. El correo pertenece a la categoría cuyo embedding sea más cercano.

5. Errores Comunes y Mejores Prácticas

Incluso los arquitectos con experiencia suelen caer en estas trampas:

1. **Tratar los embeddings como "comprensión total"**: Los modelos de embeddings capturan correlaciones estadísticas, no lógica pura. Pueden confundir la negación (ej: "Hay stock" y "No hay stock" pueden quedar cerca porque ambos tratan sobre disponibilidad).
2. **Ignorar el límite de tokens**: Cada modelo de embedding tiene una ventana de contexto. Si intentas embeber un PDF de 100 páginas de una vez, el modelo truncará el texto o perderá precisión. Aquí entra el concepto de *Chunking* (que veremos en la próxima unidad).
3. **Mezclar modelos**: **NUNCA** compares un embedding generado con OpenAI con uno generado con un modelo local de HuggingFace. Viven en "universos" (espacios vectoriales) distintos y con dimensiones diferentes. No hay relación matemática entre ellos.
4. **No considerar el "Domain Shift"**: Un modelo entrenado con textos generales de internet puede fallar miserablemente al medir similitudes en lenguaje médico altamente especializado o en código fuente muy técnico.

6. Síntesis y Próximos Pasos

Hemos pasado de ver el texto como una secuencia de caracteres a verlo como un **punto en un espacio geométrico**.

- Los **embeddings** son la traducción del lenguaje a coordenadas numéricas.
- La **similitud coseno** es nuestra herramienta para medir qué tan cerca están dos ideas.
- Esta capacidad es la base de la "Memoria a largo plazo" de cualquier sistema de IA moderno.

En la arquitectura RAG (Retrieval-Augmented Generation), los embeddings son el motor que busca la información relevante. Sin embargo, antes de poder buscar en miles de documentos, necesitamos aprender a "trocearlos" de forma inteligente. En la siguiente unidad, exploraremos las **Estrategias de Chunking**, el arte de dividir el conocimiento para que no se pierda el sentido en la traducción geométrica.

Estrategias de Chunking y preprocesamiento de documentos

Implementa pipelines de procesamiento de documentos utilizando técnicas de limpieza y Recursive Character Splitting para optimizar la ingesta en bases de datos vectoriales.

En la arquitectura RAG (Generación Aumentada por Recuperación), la calidad de la respuesta del LLM no depende solo del modelo, sino fundamentalmente de la calidad de los datos que recuperamos. Aquí es donde el **Chunking** se vuelve crítico.

¿Por qué necesitamos fragmentar?

Los modelos de lenguaje y, más importante aún, los modelos de **Embedding**, tienen límites de "ventana de contexto". Si intentas convertir un PDF de 50 páginas en un solo vector, perderás todos los

matices y detalles. El resultado será un promedio semántico tan diluido que no servirá para encontrar respuestas específicas.

El objetivo del chunking es crear **unidades de información atómicas y semánticamente coherentes**.

1. El Proceso de Limpieza (Preprocessing)

Antes de dividir, debemos limpiar. Los documentos reales (PDFs, HTMLs) suelen traer ruido:

- Encabezados y pies de página repetidos.
- Saltos de línea dobles o triples.
- Caracteres especiales resultantes de una mala extracción de tablas.
- Rutas de archivos o metadatos basura.

Por qué importa: Si un chunk contiene un pie de página en medio de una frase importante, el modelo de embedding capturará ese ruido, afectando la similitud semántica.

2. Estrategias de Chunking

A. Fixed-size (Tamaño Fijo)

Divide el texto cada X caracteres o tokens. Es el método más simple pero el más peligroso, ya que puede cortar una oración o un concepto por la mitad.

B. Recursive Character Splitting (El estándar de industria)

Es el enfoque preferido en LangChain. Intenta dividir el texto usando una jerarquía de separadores:

1. Párrafos (`\n\n`)
2. Saltos de línea (`\n`)
3. Espacios ()
4. Caracteres individuales (como último recurso)

Esto garantiza que, en la medida de lo posible, los párrafos y oraciones se mantengan unidos.

C. Overlap (Solapamiento)

Al dividir, permitimos que el final del "Chunk 1" sea el inicio del "Chunk 2".

- **Motivación:** Imagina que un concepto clave se explica en una frase que queda justo en el límite entre dos fragmentos. El overlap asegura que el contexto se preserve en ambos, facilitando la recuperación sin importar por dónde se "cortó" el papel virtual.

Errores Comunes

1. **Chunks demasiado pequeños:** Generan ruido y pierden contexto global. El LLM tendrá dificultades para entender de qué trata el fragmento.
2. **Ignorar los límites de tokens:** Muchos desarrolladores miden en *caracteres*, pero los modelos cobran y limitan por *tokens*. Siempre valida tus chunks con un tokenizador real (como `tiktoken`).

3. **No limpiar el texto:** "Basura entra, basura sale". Un chunk mal preprocesado reduce la precisión de la búsqueda vectorial.

Síntesis

Un buen chunking es un equilibrio entre:

1. **Granularidad:** Fragmentos pequeños para precisión.
2. **Contexto:** Fragmentos lo suficientemente grandes (u con overlap) para ser comprensibles.
3. **Limpieza:** Datos normalizados para embeddings precisos.

Persistencia local con ChromaDB: operaciones CRUD

Aprende a gestionar el ciclo de vida de los datos en ChromaDB local, implementando persistencia en disco y operaciones CRUD para sistemas de memoria en IA.

En el desarrollo de sistemas de IA, los **embeddings** (vectores que representan el significado del texto) son volátiles si solo viven en la memoria RAM. Para aplicaciones reales, necesitamos que el conocimiento del sistema "sobreviva" al reinicio del servidor y pueda actualizarse sin reindexar todo desde cero. Aquí es donde entra **ChromaDB**.

¿Por qué ChromaDB local?

ChromaDB es una base de datos vectorial de código abierto diseñada para ser simple. En un entorno de desarrollo o para aplicaciones de escritorio/privadas, su modo **Local Persistente** es ideal:

- No requiere levantar un contenedor Docker complejo (por defecto).
- Guarda los datos en archivos `.parquet` y `bin` en una carpeta de tu elección.
- Permite operaciones CRUD (Create, Read, Update, Delete) sobre el "conocimiento" de tu agente.

Operaciones CRUD en el Espacio Vectorial

A diferencia de una base de datos SQL tradicional, el CRUD en una DB vectorial tiene matices importantes:

1. **Create (Añadir/Upsert):** No solo guardamos texto, sino su vector asociado. Si el texto cambia, el vector **debe** recalcularse. El uso de `upsert` es una mejor práctica: si el ID ya existe, actualiza; si no, lo crea.
2. **Read (Get vs Query):**
 - `get()` : Busca por IDs exactos o filtros de metadatos (como un SELECT tradicional).
 - `query()` : Busca por **similitud semántica**. Es el corazón de RAG.
3. **Update:** Actualiza los metadatos o el documento (disparando una nueva vectorización).
4. **Delete:** Elimina vectores que ya no son relevantes para el contexto.

El Rol de los Metadatos

Los metadatos son "etiquetas" que adjuntamos a los vectores (ej: `{"autor": "Juan", "fecha": "2024-05", "source": "pdf_v1"}`). Son cruciales para:

- Filtrar resultados antes de la búsqueda vectorial.
- Rastrear el origen de la información que usa el LLM (citación de fuentes).

Errores Comunes

- **ID no determinista:** Usar IDs aleatorios hace imposible el `update` eficiente. Se recomienda usar hashes del contenido o IDs estructurados.
- **Desajuste de Embeddings:** Intentar consultar una base de datos local con un modelo de embeddings diferente al que se usó para crearla. Los resultados serán ruido aleatorio.
- **Ignorar la persistencia:** No inicializar el cliente con una ruta de disco, causando pérdida de datos al cerrar el script.

Sistema de recuperación semántica local (RAG)

Tercer checkpoint del programa enfocado en la construcción de un sistema RAG (Retrieval-Augmented Generation) funcional que integra memoria vectorial local con lógica de generación fundamentada.

¿Qué debes construir?

Debes entregar un script o notebook de Python que implemente un flujo **End-to-End de RAG**. El sistema debe ser capaz de recibir una consulta del usuario, buscar la información relevante en una base vectorial previamente poblada y generar una respuesta que utilice *exclusivamente* esa información.

Componentes a entregar:

1. **Módulo de Ingesta (Setup):** Una función que tome un conjunto de documentos (pueden ser archivos `.txt` o `.md` simples), los fragmente (*chunking*) y los persista en una colección de **ChromaDB**.
2. **Capa de Recuperación (Retriever):** Una lógica que convierta la pregunta del usuario en un embedding y recupere los fragmentos más relevantes.
3. **Generación Grounded:** Una cadena de **LangChain (LCEL)** que reciba los documentos recuperados y la pregunta, generando una respuesta. El prompt debe instruir al modelo a decir "No lo sé" si la respuesta no está en el contexto.

Pasos sugeridos

1. **Puebla tu "Cerebro":** Elige 3 o 4 archivos de texto sobre un tema específico (ej. manuales técnicos, apuntes de clase o normativas). Usa las técnicas de *RecursiveCharacterTextSplitter* que vimos en la Unidad 2 para procesarlos.
2. **Configura ChromaDB:** Inicializa el cliente persistente en una carpeta local (ej. `./vectorstore`). Asegúrate de usar el mismo modelo de embeddings para indexar y para consultar.
3. **Construye el Prompt de Sistema:** Diseña un prompt que actúe como un "filtro de veracidad". Ejemplo: "Eres un asistente técnico. Responde solo basándote en el CONTEXTO proporcionado. Si la respuesta no está allí, di que no tienes acceso a esa información."
4. **Crea la Cadena LCEL:** Une el retriever con el transformador de documentos y el modelo de lenguaje. Recuerda que el output debe pasar por un `PydanticOutputParser`.

Errores comunes a evitar

- **El "Contexto Infinito":** No intentes pasar 50 fragmentos al LLM. Esto causará errores de límite de tokens o degradación de la atención del modelo (*Lost in the Middle*). Mantén un `top_k` de entre 3 y 5.
- **Embeddings no coincidentes:** Es el error #1. Si indexas tus documentos con `OpenAIEmbeddings` y consultas con `HuggingFaceEmbeddings`, la distancia vectorial no tendrá sentido y los resultados serán aleatorios.
- **Falta de persistencia:** Asegúrate de que el script verifique si la base de datos ya existe antes de volver a indexar todo, optimizando así el tiempo y costo de ejecución.

Qué entregás y en qué formato

- **Tipo:** Código — un repositorio de GitHub.
- **Artefacto concreto:** repo con el script de ingesta (chunking + ChromaDB), el script/notebook de la cadena RAG asíncrona, un dataset de ejemplo (`.txt` / `.md`) y un `README.md`.
- **Qué NO hace falta:** no es un documento de análisis; el `README.md` alcanza como documentación.

Pasos para la entrega

1. Crea un entorno virtual e instala las dependencias necesarias: `langchain`, `chromadb`, `openai` (o el provider seleccionado), y `pydantic`.
2. Implementa un script de carga que lea archivos de una carpeta `/data`, aplique un chunking estratégico (mínimo 500 tokens con 50 de overlap) y los guarde en una instancia local de ChromaDB.
3. Define una clase o función asíncrona `get_rag_response(query: str)` que: a. Realice una búsqueda de similitud en ChromaDB. b. Construya un prompt incluyendo los fragmentos recuperados. c. Llame al LLM de forma asíncrona. d. Parsee la respuesta a un modelo Pydantic que incluya el texto y las referencias.
4. Realiza dos pruebas: una pregunta cuya respuesta esté en los documentos y una "pregunta trampa" cuya respuesta no esté, verificando que el modelo no alucine.
5. Sube tu código asegurándote de no incluir llaves de API (usa variables de entorno `.env`).

Entregable

Formato: repositorio

Repositorio de GitHub que contenga: 1) Script de ingesta de documentos, 2) Script/Notebook con la cadena RAG asíncrona, 3) Un dataset de ejemplo (txt/md) y 4) Un archivo README explicando cómo ejecutar el sistema.

Objetivos de aprendizaje

- Diseñar una arquitectura RAG completa integrando componentes de recuperación y generación.

- Implementar persistencia vectorial local utilizando ChromaDB y LangChain.
- Garantizar el 'grounding' de las respuestas del LLM mediante el diseño de prompts y contextos restringidos.
- Validar salidas de sistemas RAG utilizando esquemas de Pydantic para asegurar la trazabilidad de fuentes.

Rúbrica de evaluación

Evaluación de la implementación de un flujo RAG completo, desde la ingesta de documentos en ChromaDB hasta la respuesta validada con Pydantic y lógica asíncrona.

Criterio	Descripción	Peso
Estrategia de Chunking e Ingesta	Capacidad para procesar documentos de la carpeta /data aplicando técnicas de fragmentación que preserven el contexto semántico.	25%
Arquitectura Asíncrona y Recuperación	Implementación de la función get_rag_response utilizando patrones asíncronos para la consulta y el LLM.	35%
Validación con Pydantic y Esquemas	Uso de modelos Pydantic para garantizar que la salida del sistema sea estructurada y contenga las fuentes.	20%
Robustez y Evidencia de Pruebas	Verificación del comportamiento del sistema ante preguntas fuera de contexto y seguridad de las credenciales.	20%

Total: 100 pts | Aprobación: 70 pts

Escalabilidad documental: RAG avanzado y pinecone

Optimizar la escalabilidad y precisión de sistemas de recuperación utilizando Pinecone y técnicas avanzadas de procesamiento documental.

Pinecone Serverless: persistencia vectorial en la nube

Aprende a configurar una infraestructura de base de datos vectorial profesional en la nube con Pinecone Serverless y Python.

En el módulo anterior trabajamos con **ChromaDB**, una base de datos vectorial excelente para desarrollo local y prototipado rápido. Sin embargo, cuando movemos sistemas de IA a producción, nos enfrentamos a desafíos de **escalabilidad, persistencia distribuida y alta disponibilidad**.

Aquí es donde entra **Pinecone**.

¿Por qué Pinecone en el Stack de un AI Architect?

Pinecone es una base de datos vectorial nativa de la nube, diseñada específicamente para manejar millones (o miles de millones) de vectores con latencias de milisegundos. A diferencia de las bases de datos locales:

1. **Arquitectura Serverless:** No necesitas gestionar nodos ni memoria. Pagas por lo que usas y el almacenamiento se escala automáticamente.
2. **Separación de Computación y Almacenamiento:** Permite que las búsquedas sean rápidas sin importar cuántos datos tengas guardados.
3. **Persistencia Garantizada:** Tus datos viven en la infraestructura administrada de Pinecone, no en tu contenedor local.

Conceptos Clave de Infraestructura

Antes de programar, debemos entender los objetos fundamentales:

- **Index (Índice):** Es el contenedor principal. Define la **dimensión** de los vectores (que debe coincidir con tu modelo de embeddings) y la **métrica de similitud** (Coseno, Producto Punto o Euclidiana).
- **Nombre de Espacio (Namespace):** Permite segmentar un solo índice en múltiples particiones lógicas. Es vital para implementar multi-tenancy (separar datos de diferentes clientes) o separar entornos (dev, staging, prod) dentro de un mismo índice.
- **Metadatos:** Información adjunta al vector (JSON) que permite realizar filtrados adicionales durante la búsqueda.

El Flujo de Trabajo (Pipeline)

En un entorno profesional, Pinecone no vive solo. Se integra en un pipeline:

1. **Ingesta:** Los documentos se fragmentan (Chunking), se convierten a vectores (Embedding) y se suben (`upsert`) a Pinecone.
2. **Recuperación (Retrieval):** La consulta del usuario se vectoriza, se busca el "vecino más cercano" en Pinecone filtrando por metadatos, y el resultado alimenta al LLM.

Errores Comunes de Arquitectura

- **Mismatch de Dimensiones:** Intentar insertar un vector de 1536 dimensiones en un índice configurado para 768. Esto lanzará un error inmediato.
- **Ignorar los Namespaces:** Mezclar datos de prueba con producción o datos de diferentes usuarios en el mismo espacio lógico, comprometiendo la seguridad y la claridad de los resultados.
- **Elección Errónea de Métrica:** Usar similitud euclidiana cuando el modelo de embeddings fue entrenado específicamente para optimizar la similitud coseno.

En esta unidad, aprenderás a automatizar la creación de esta infraestructura usando el SDK de Python.

Ingesta masiva y gestión de metadatos avanzados

Aprende a construir pipelines de datos robustos utilizando procesamiento por lotes y metadatos enriquecidos para búsquedas vectoriales filtradas y seguras.

En un sistema RAG de producción, no basta con "subir archivos". La capacidad de respuesta y la precisión del sistema dependen crucialmente de cómo estructuramos los datos en el momento de la ingesta. Cuando pasamos de 100 documentos a 1,000,000, el **filtrado por metadatos** y el **procesamiento por lotes (batching)** dejan de ser extras para convertirse en requisitos de arquitectura.

El Corazón de la Escalabilidad: Ingesta Distribuida y Batching

Ingresar datos uno por uno (`vector a vector`) es ineficiente debido al overhead de red y los límites de la API de Pinecone. La estrategia correcta es el **Batch Upsert**.

- **¿Por qué batches?:** Reduce latencia de red y aprovecha el paralelismo interno de la base de datos vectorial.
- **Tamaño ideal:** Generalmente, batches de entre 100 y 200 vectores son el "punto dulce" entre velocidad y estabilidad.

Metadatos: El Índice Selectivo para la IA

Si buscas "factura de Amazon" en un mar de 10 millones de documentos, la búsqueda semántica pura puede darte resultados de "facturas de Azure" o "recibos de Walmart" porque son semánticamente similares. Los **metadatos** permiten aplicar un filtro rígido (Hard Filter) antes o durante la búsqueda semántica.

En Pinecone, los metadatos se almacenan junto al vector y permiten consultas SQL-like. Algunos casos de uso avanzados incluyen:

1. **Multi-tenancy:** Filtrar por `user_id` o `org_id` para asegurar que un usuario nunca vea datos de otro.
2. **Temporalidad:** Filtrar por `year > 2023` para evitar información obsoleta.
3. **Categorización:** Limitar el contexto a `department == 'legal'`.

El Riesgo del "Schema Drift"

Un error común es no estandarizar los metadatos. Si un proceso guarda la fecha como `ingest_date` y otro como `date_created`, tus filtros fallarán. Como arquitecto, debes definir una **interfaz de metadatos** estricta.

Ejemplo de Filtro en Pinecone (Conceptual)

CÓDIGO

```
# Queremos buscar similitud solo en documentos de la categoría 'legal'
index.query(
    vector=[0.1, 0.2, ...],
    top_k=10,
    filter={
        "category": {"$eq": "legal"},
        "status": {"$in": ["reviewed", "final"]}
    }
)
```

Este enfoque reduce drásticamente el espacio de búsqueda, mejorando la latencia y eliminando el "ruido" de categorías irrelevantes.

Flashcards: Métricas y recuperación híbrida

Unidad de repaso sobre las métricas críticas de evaluación para sistemas de búsqueda y los conceptos fundamentales de la arquitectura de recuperación híbrida.

Flashcards: Métricas y recuperación híbrida

En sistemas RAG de alta escala, la calidad de la respuesta depende totalmente de la relevancia de los documentos recuperados. Como arquitectos de IA, no basta con "conectar" una base de datos vectorial; debemos entender cómo evaluar y optimizar el proceso de recuperación para garantizar que no estemos entregando "ruido" al LLM.

Esta unidad se centra en los pilares fundamentales para medir y mejorar la recuperación:

1. **Métricas de Calidad:** Entender cuándo estamos recuperando documentos útiles (Precisión) y cuándo estamos omitiendo documentos clave (Recall).

2. **Recuperación Híbrida:** Superar las limitaciones de los embeddings (semántica) combinándolos con la robustez de la búsqueda léxica (BM25).
3. **Fusión y Re-ranking:** Estrategias como RRF para combinar resultados heterogéneos y modelos de re-ranking (cross-encoders) para un refinamiento final de alta precisión.

Cómo utilizar estas tarjetas

- **Intenta responder mentalmente** antes de hacer clic para voltear la tarjeta.
- **Visualiza el escenario:** Si una tarjeta menciona **Precision**, piensa en un sistema de búsqueda donde el usuario espera resultados exactos y no quiere basura.
- **Enfócate en la relación:** No memorices las fórmulas como algo abstracto; entiende qué fenómeno del mundo real intenta capturar cada métrica.

¡Comencemos el repaso!

01 ¿Cuál es la diferencia fundamental entre Precision y Recall en un sistema RAG y por qué ambos son críticos?

La **Precisión** mide cuánto de lo que recuperó el sistema es realmente relevante (evita resultados irrelevantes). El **Recall** mide qué proporción de todos los documentos relevantes existentes fue encontrada (evita dejar información útil fuera). En RAG, un recall bajo causa alucinaciones por falta de contexto, mientras que una precisión baja introduce ruido que confunde al LLM.

02 ¿Por qué los motores de búsqueda modernos combinan BM25 con búsqueda semántica (embeddings)?

BM25 se basa en coincidencias exactas de términos (frecuencia de palabras, rareza). Es excelente para nombres propios, códigos de error o términos técnicos. Los **Embeddings** capturan el significado conceptual y la intención del usuario, manejando sinónimos y variaciones lingüísticas que BM25 ignoraría. La búsqueda híbrida captura lo mejor de ambos mundos.

03 ¿Qué es el RRF y por qué es la estrategia preferida para fusionar resultados en una búsqueda híbrida?

El **Reciprocal Rank Fusion (RRF)** combina rankings de diferentes fuentes (ej. BM25 y Vectores) basándose en las posiciones (rank) de los documentos en lugar de sus puntuaciones absolutas. Es robusto porque no requiere normalizar puntuaciones que provienen de sistemas con escalas totalmente distintas.

04 ¿Cuál es el propósito del Re-ranking en un pipeline de recuperación y en qué se diferencia del retrieval inicial?

El **Re-ranking** es una segunda etapa donde un modelo (generalmente un *cross-encoder* más pesado) evalúa con mayor precisión el subconjunto de documentos devueltos por la primera etapa de recuperación. Permite mayor exactitud sin sacrificar la latencia de todo el sistema, ya que se aplica solo a un pequeño número de candidatos (top-k).

05 Defina Recall@k y explique su importancia específica en el contexto de un sistema RAG.

Recall@k mide si al menos un documento relevante está incluido dentro de los primeros k resultados recuperados. Es la métrica reina para evaluar si nuestro sistema es capaz de "entregar el contexto necesario" al LLM de forma consistente.

06 ¿Cuál es el error conceptual más común al implementar una búsqueda híbrida "a mano"?

Un error frecuente es intentar sumar directamente las puntuaciones (scores) de BM25 y de similitud vectorial. Esto no funciona porque sus escalas son heterogéneas (ej. un score de 20 en BM25 no equivale a una similitud coseno de 0.8). Es indispensable normalizar o usar algoritmos de fusión como RRF.

07 ¿Qué caracteriza a un Cross-encoder y por qué no se usa para la etapa de retrieval inicial?

El **Cross-encoder** procesa el par (query, documento) conjuntamente desde el inicio para calcular la relevancia. Es extremadamente preciso porque entiende interacciones profundas entre palabras, pero es demasiado lento para buscar en millones de documentos, por lo que se reserva estrictamente para la fase de re-ranking.

08 ¿Qué nos indica el F1-Score en la evaluación de un sistema de recuperación?

El **F1-Score** es la media armónica entre precisión y recall. Se utiliza para encontrar un equilibrio único que resuma el rendimiento del sistema. Es especialmente útil en datasets donde no podemos permitirnos ni demasiada relevancia baja ni demasiada omisión de datos.

Sistema RAG escalable en la nube con Pinecone

Guía detallada para la pre-entrega del Módulo 4, donde el estudiante debe escalar su arquitectura RAG de un entorno local a Pinecone Cloud, implementando búsqueda híbrida y evaluación de métricas.

Qué construir

Debes entregar un **Módulo de Recuperación Escalable** integrado en un repositorio de código. El artefacto principal es un servicio (o conjunto de scripts organizados) en Python que ejecute el flujo completo de un sistema RAG en la nube.

Componentes obligatorios:

1. **Pipeline de Ingesta en Pinecone:** Un script que tome un conjunto de documentos (PDFs, Markdown o JSON), los procese y los suba a un índice de Pinecone Serverless utilizando metadatos avanzados (fuente, página, etiquetas de categoría).
2. **Recuperador Híbrido (Hybrid Retriever):** Una implementación que combine búsqueda por similitud de vectores con búsqueda léxica (BM25) para mejorar la precisión en términos técnicos o nombres propios.
3. **Script de Evaluación:** Una utilidad que calcule al menos dos métricas fundamentales (Precision@k y Recall@k) utilizando un pequeño "Golden Set" de preguntas y respuestas de prueba.

Pasos sugeridos

1. **Preparación de Infraestructura:** Crea un índice *Serverless* en Pinecone (usa la dimensión 1536 si usas `OpenAI text-embedding-3-small`).
2. **Ingesta Inteligente:** No subas el texto plano. Crea un esquema donde guardes el texto original dentro de los metadatos de Pinecone para evitar consultas adicionales a una base de datos relacional.
3. **Configuración de LangChain:** Utiliza el `PineconeVectorStore` de LangChain o el SDK nativo de Pinecone para configurar el motor de búsqueda.
4. **Implementación BM25:** Configura un recuperador de LangChain que use `BM25Retriever` y combínalo con el de Pinecone usando un `EnsembleRetriever`.
5. **Evaluación Local:** Crea un pequeño archivo JSON con pares `{"pregunta": "...", "documento_id_esperado": "..."}` y mide cuántos de esos documentos aparecen efectivamente en el Top-5 recuperado.

Errores comunes a evitar

- **Mismatch de Dimensiones:** Intentar subir embeddings de 1536 dimensiones a un índice configurado con 512 o 768.
- **Ignorar el Namespace:** En aplicaciones multi-inquilino o con distintos tipos de datos, no usar namespaces en Pinecone hará que la búsqueda sea ruidosa y lenta.
- **Subestimar el Chunking:** Chunks muy pequeños pierden el contexto semántico; muy grandes diluyen la precisión del embedding. Busca un punto medio (~500-800 tokens).

Qué entregás y en qué formato

- **Tipo:** Código — un repositorio de GitHub.
- **Artefacto concreto:** repo con el pipeline de ingesta a Pinecone, el recuperador híbrido (BM25 + vectorial) y `evaluate.py` (Precision@k y Recall@k). El `README.md` debe incluir los pasos para replicar el índice.
- **Qué NO hace falta:** no hay PDF; el reporte de métricas se imprime en consola y se resume en el `README.md`.

Pasos para la entrega

1. **Configuración de variables:** Crea un archivo `.env` con `PINECONE_API_KEY`, `OPENAI_API_KEY` (o Anthropic) e `INDEX_NAME`.
2. **Setup de Pinecone:** Escribe un script de inicialización que verifique si el índice existe y lo cree si es necesario (modo Serverless).
3. **Pipeline de Ingesta:**
 - Carga un dataset de documentos técnicos (puedes usar la documentación de una librería de Python).
 - Divide en chunks usando un `RecursiveCharacterTextSplitter`.
 - Genera embeddings e insértalos en Pinecone incluyendo el contenido y la fuente en la metadata.
4. **Implementación del Recuperador:**
 - Crea una clase `RAGSystem` que encapsule un `EnsembleRetriever`.
 - El sistema debe recibir una consulta y devolver los top-5 documentos combinando resultados léxicos y semánticos.
5. **Evaluación:**
 - Crea un script `evaluate.py`.
 - Define un benchmark de 5 preguntas donde conozcas de antemano el documento fuente.
 - Ejecuta las consultas y calcula:
 - **Recall@5:** ¿Está el documento correcto entre los 5 recuperados?
 - **Precision@5:** ¿Qué porcentaje de los 5 recuperados son realmente útiles?

6. **Reporte:** Imprime en consola un breve resumen de los resultados de evaluación.

Entregable

Formato: repositorio

Repositorio de GitHub que contenga el pipeline de ingesta, el recuperador híbrido y el script de evaluación de métricas. El README debe incluir instrucciones para replicar el índice de Pinecone.

Objetivos de aprendizaje

- Implementar una base de datos vectorial en la nube con Pinecone Serverless.
- Diseñar estrategias de ingesta masiva con metadatos estructurados para filtrado.
- Construir un sistema de recuperación híbrida combinando BM25 y búsqueda de vectores.
- Evaluar el rendimiento del sistema de recuperación mediante métricas de Precision y Recall.

Rúbrica de evaluación

Esta evaluación mide la capacidad del estudiante para diseñar e implementar un pipeline de recuperación de información escalable en la nube, integrando técnicas de búsqueda híbrida y evaluando cuantitativamente el rendimiento del sistema mediante métricas de recuperación.

Criterio	Descripción	Peso
Configuración e Infraestructura Cloud (Pinecone)	Evaluación de la correcta inicialización del índice serverless y la gestión de variables de entorno de forma segura.	15%
Pipeline de Ingesta y Gestión de Metadatos	Capacidad para procesar documentos técnicos, fragmentarlos estratégicamente y persistirlos con metadatos útiles.	25%
Implementación del Recuperador Híbrido	Diseño de la clase RAGSystem utilizando EnsembleRetriever para combinar búsqueda semántica y léxica.	35%
Evaluación Cuantitativa de Métricas	Implementación del script evaluate.py para calcular Recall@5 y Precision@5 basado en un benchmark.	25%

Total: 100 pts | Aprobación: 70 pts

Razonamiento autónomo: introducción de agentes con LangGraph

Desarrollar agentes capaces de razonar cíclicamente y utilizar herramientas externas mediante el paradigma de grafos de LangGraph.

Arquitectura de grafos: nodos, aristas y estado en LangGraph

Aprende a estructurar flujos de IA avanzados utilizando LangGraph, dominando los pilares de nodos, aristas y estados tipados para crear sistemas autónomos con ciclos de razonamiento.

Bienvenido al Módulo 5. Hasta ahora, has construido cadenas lineales con LCEL y has implementado arquitecturas RAG para dotar a tus modelos de una base de conocimientos. Sin embargo, en el mundo real, los problemas rara vez son lineales. Los sistemas de IA más avanzados no solo "responden", sino que **razonan, iteran y corrigen sus propios errores**.

Imagina un sistema de soporte técnico. No es una simple secuencia de pasos; es un flujo donde, dependiendo de la complejidad, el caso puede resolverse, pedir más información al usuario o escalar a un humano. Intentar programar esto con cadenas `if/else` interminables o loops de chat básicos resulta en un código frágil, difícil de depurar y casi imposible de escalar.

Aquí es donde entra **LangGraph**. En esta unidad, aprenderás a pensar en términos de **Grafos de Estado**, una evolución necesaria para construir agentes autónomos con principios de ingeniería de software.

A dónde vas con este módulo (leé esto primero)

Todo lo de este módulo apunta a **una** entrega: la **Pre-entrega 5 — un agente de razonamiento cíclico con LangGraph** (un repo con un `StateGraph` que usa herramientas, razona en varios pasos y recuerda la conversación con un checkpointer).

Para lograrlo, de toda la teoría que sigue solo necesitás dominar **4 conceptos núcleo**:

1. **Estado (State)**: la "memoria de trabajo" del grafo (`TypedDict` / `MessagesState`).
2. **Nodos y aristas condicionales**: los pasos y las decisiones ("¿sigo o termino?").
3. **Ciclo ReAct**: el agente llama a una herramienta, observa el resultado y decide el siguiente paso.
4. **Persistencia (checkpointer)**: `SqliteSaver` para que la conversación no se pierda.

Si entendés esos 4, ya podés resolver la pre-entrega. El resto de la unidad es **profundización**: útil para dominar el tema, pero no imprescindible para entregar. Leela con calma, sin sentir que "tenés que memorizar todo".

1. ¿Por qué LangGraph? El paso de Cadenas a Grafos

En los módulos anteriores, utilizamos LangChain para crear secuencias de pasos (cadenas). Estas cadenas son excelentes para flujos de trabajo predecibles: *Input -> Prompt -> LLM -> Output*.

Sin embargo, los **agentes autónomos** requieren ciclos. Un agente necesita observar el resultado de una herramienta y decidir: "¿He terminado?" o "¿Debo intentar otra cosa?". Las cadenas de LangChain (DAGs por naturaleza en su versión básica) dificultan la implementación de estos ciclos de forma limpia.

LangGraph extiende LangChain permitiendo la creación de grafos que pueden contener ciclos. Pero lo más importante no es el ciclo en sí, sino la **gestión del estado**. En un sistema complejo, necesitamos una "memoria de trabajo" que persista mientras el agente navega por diferentes procesos.

2. El Estado: La Memoria Compartida del Grafo

El concepto más fundamental en LangGraph es el **Estado (State)**. Si visualizamos el grafo como una línea de ensamblaje en una fábrica, el Estado es el producto que se mueve por la cinta transportadora. Cada estación (nodo) puede leer el producto, modificarlo o añadirle piezas antes de pasarlo a la siguiente.

Definición mediante TypedDict

En Python 3.12+, definimos el estado utilizando `TypedDict`. Esto nos permite tener un esquema claro y tipado de qué datos están "vivos" en nuestra aplicación.

CÓDIGO

```
from typing import Annotated, TypedDict, List
from langchain_core.messages import BaseMessage

# Definimos el esquema de nuestra "memoria de trabajo"
class AgentState(TypedDict):
    # El historial de mensajes es el estado más común
    # Usamos Annotated y un "reducer" para decidir cómo se actualiza este campo
    messages: Annotated[List[BaseMessage], add_messages]
    # Podemos añadir campos personalizados para lógica de negocio
    is_escalated: bool
    current_step: str
```

El concepto de Reducers

Un error común es pensar que un nodo sobrescribe todo el estado. En realidad, los nodos devuelven **actualizaciones**. Si el estado tiene 10 campos y un nodo devuelve un diccionario con 1 campo, solo ese campo se actualiza.

¿Qué pasa si queremos que una lista de mensajes crezca en lugar de sobrescribirse? Usamos un **Reducer**. En el ejemplo de arriba, `add_messages` es una función que le dice a LangGraph: "No borres los mensajes anteriores, anexa los nuevos a la lista".

3. Nodos: Las Unidades de Trabajo

Los **Nodos** son simplemente funciones de Python (síncronas o `async`) que toman el estado actual como entrada y devuelven una actualización del estado.

Un nodo no tiene por qué ser un LLM. Puede ser:

1. **Un llamado a un LLM:** Procesa el historial de mensajes y decide qué hacer.
2. **Una herramienta (Tool):** Realiza una consulta a una base de datos o API.
3. **Lógica de negocio:** Valida un JSON, formatea un texto o cambia una bandera en el estado.

Anatomía de un Nodo

CÓDIGO

```
async def call_model(state: AgentState):
    # 1. Recuperamos información del estado
    messages = state['messages']

    # 2. Realizamos la operación (ej: llamado al LLM)
    response = await model.ainvoke(messages)

    # 3. Devolvemos la actualización
    # LangGraph se encarga de fusionar esto con el estado global
    return {"messages": [response]}
```

Principio de diseño: Mantén los nodos pequeños y con una única responsabilidad. Esto facilita las pruebas unitarias y la observabilidad. En LangGraph, un nodo es una unidad de software testeable, no una "caja negra mágica".

4. Aristas: El Control de Flujo

Si los nodos son las estaciones de trabajo, las **Aristas (Edges)** son las reglas que definen hacia dónde se mueve el producto después de salir de una estación.

A. Aristas Directas (Normal Edges)

Simplemente conectan el Nodo A con el Nodo B. "Siempre que termines de procesar en A, ve a B".
Ejemplo: `workflow.add_edge("limpiar_texto", "analizar_sentimiento")`.

B. Aristas Condicionales (Conditional Edges)

Permiten que el flujo se bifurque basándose en la lógica. Se apoyan en una **función de enrutamiento**.

CÓDIGO

```
def route_after_analysis(state: AgentState):
    # Si el sentimiento es negativo, escalamos a un humano
    if state["sentiment"] == "negative":
        return "human_support"
    # Si no, terminamos
    return "end"

# En la definición del grafo:
workflow.add_conditional_edges(
    "analizar_sentimiento",
    route_after_analysis,
    {
        "human_support": "nodo_humano",
        "end": END
    }
)
```

Aquí es donde ocurre la "magia" de los agentes autónomos. Una arista condicional puede devolver el flujo a un nodo anterior, creando un **ciclo de reflexión**. Por ejemplo, un nodo "Validador" podría decidir que la respuesta del LLM no es lo suficientemente buena y enviarla de vuelta al nodo "Escritor" para una corrección.

5. El Flujo de Trabajo (Compiled Graph)

Una vez definidos los nodos y las aristas, el grafo debe ser compilado. Al compilarlo, LangGraph valida que:

1. Haya un punto de entrada (`START`).
2. Todos los caminos tengan una salida posible o lleguen al nodo especial `END` .
3. No existan nodos aislados.

Metáfora del "Libro de Elige tu propia aventura"

Imagina que cada nodo es una página del libro.

- El **Estado (State)** es lo que el lector recuerda (su inventario, su salud).
- Los **Nodos** son el contenido de la página que el lector lee.
- Las **Aristas** son las instrucciones al final de la página: "Si decides abrir la puerta, ve a la página 45; si decides correr, ve a la página 12".
- Las **Aristas Condicionales** son instrucciones basadas en el inventario: "Si tienes la llave dorada (estado), ve a la página 100; si no, ve a la página 20".

6. Errores Comunes y Mejores Prácticas

Error 1: Cargar el Estado con datos innecesarios

Un error frecuente es intentar meter objetos pesados (como clientes de bases de datos o conexiones de red) dentro del objeto `State`.

Nota

Mejor práctica: El estado debe contener solo datos que necesiten persistir o ser compartidos entre nodos. Los objetos de infraestructura deben pasarse a través de la configuración o ser parte de la clase que envuelve al grafo.

Error 2: Nodos demasiado grandes

Si un nodo hace un llamado al LLM, procesa el resultado, limpia los datos y guarda en la base de datos, estás perdiendo el beneficio de la granularidad de los grafos.

Nota

Mejor práctica: Divide las operaciones. Un nodo para el LLM y un nodo para el post-procesamiento. Esto permite insertar aristas condicionales en medio (ej. "¿Logró el LLM extraer los datos correctamente o necesitamos reintentar?").

Error 3: Olvidar el Reducer en listas de mensajes

Si defines `messages: List[BaseMessage]` en tu `TypedDict` sin un reducer, cada vez que un nodo devuelva `{"messages": [new_msg]}`, la lista anterior desaparecerá.

Nota

Mejor práctica: Usa siempre `Annotated[List[BaseMessage], add_messages]` para flujos de chat convencionales.

7. Ejemplo resuelto paso a paso: un grafo de soporte con ciclo y escalado

El entregable de esta unidad te pide *diseñar el diagrama*. Acá ves su **contraparte en código, completa y ejecutable**: un grafo mínimo que busca una solución, reintenta si no la encuentra (ciclo) y escala a un humano tras 3 intentos. Corre sin API keys, porque simulamos el nodo de búsqueda con lógica simple para que puedas ver la mecánica del grafo.

```

from typing import TypedDict
from langgraph.graph import StateGraph, START, END

# 1. El Estado: la "memoria de trabajo" que fluye por el grafo
class SupportState(TypedDict):
    query: str
    attempts: int
    solved: bool
    escalated: bool

# 2. Nodo: busca en la base de conocimiento (simulado)
def search_knowledge(state: SupportState) -> dict:
    intento = state["attempts"] + 1
    encontrado = intento >= 2          # simulamos que recién resuelve en el 2º
    intento
    print(f"[search] intento {intento} -> {'resuelto' if encontrado else 'sin resultado'}")
    return {"attempts": intento, "solved": encontrado}

# 3. Nodo: escala a un humano
def human_support(state: SupportState) -> dict:
    print("[human] caso escalado a soporte humano")
    return {"escalated": True}

# 4. Arista condicional: decide el camino según el estado
def route(state: SupportState) -> str:
    if state["solved"]:
        return "end"
    if state["attempts"] >= 3:          # corta el ciclo para no quedar infinito
        return "escalate"
    return "retry"                     # vuelve a 'search' -> esto crea el CICLO

# 5. Construcción del grafo
workflow = StateGraph(SupportState)
workflow.add_node("search", search_knowledge)
workflow.add_node("human", human_support)

workflow.add_edge(START, "search")
workflow.add_conditional_edges(
    "search",
    route,
    {"end": END, "escalate": "human", "retry": "search"},
)
workflow.add_edge("human", END)

app = workflow.compile()

# 6. Ejecución
final = app.invoke(
    {"query": "No puedo iniciar sesión", "attempts": 0, "solved": False,
    "escalated": False}

```

```
)  
print(final)
```

Salida esperada:

[search] intento 1 -> sin resultado [search] intento 2 -> resuelto {'query': 'No puedo iniciar sesión', 'attempts': 2, 'solved': True, 'escalated': False}

Qué pasó, paso a paso:

1. **El Estado (SupportState)** arranca con `attempts=0` . Cada nodo devuelve solo los campos que cambia (una *actualización*), no el estado completo; LangGraph se encarga de fusionarlo.
2. **START → search** : el primer intento no encuentra solución (`solved=False`).
3. **La arista condicional route** lee el estado: como no está resuelto y aún no llegó a 3 intentos, devuelve `"retry"` , que vuelve al nodo `search` . **Ese retorno a un nodo anterior es el ciclo** que diferencia a LangGraph de una cadena lineal.
4. En el segundo intento `solved=True` , así que `route` devuelve `"end"` y el grafo termina sin escalar.
5. Si la búsqueda nunca resolviera, al llegar a `attempts >= 3` la ruta sería `"escalate"` → nodo `human` → `escalated=True` → END. Probalo cambiando la condición `encontrado` a `False` para ver ese camino.

Cuando armes el diagrama del entregable, este código es el "esqueleto" que estás representando: nodos = cajas, `add_conditional_edges` = la bifurcación, y el `"retry"` = la flecha que vuelve hacia atrás.

Segundo ejemplo resuelto: bucle de reflexión (escritor ↔ crítico)

El primer ejemplo usó un ciclo de reintento. Este muestra el otro patrón cíclico clásico de los agentes: el **bucle de reflexión**, donde un nodo "escritor" genera y un nodo "crítico" decide si está lo bastante bueno o si hay que revisar otra vez. Corre sin API keys (simulamos que la calidad mejora en cada vuelta).

```

from typing import TypedDict
from langgraph.graph import StateGraph, START, END

# 1. Estado: el borrador, su puntaje de calidad y cuántas veces lo revisamos
class WriteState(TypedDict):
    tema: str
    borrador: str
    score: int
    revisiones: int

# 2. Nodo escritor: genera/mejora el borrador (simulado: suma 3 puntos por vuelta)
def escribir(state: WriteState) -> dict:
    nuevo_score = state["score"] + 3
    print(f"[escritor] revisión {state['revisiones'] + 1} -> score {nuevo_score}")
    return {
        "borrador": f"Borrador sobre '{state['tema']}' (calidad {nuevo_score}/10)",
        "score": nuevo_score,
        "revisiones": state["revisiones"] + 1,
    }

# 3. Nodo crítico: no modifica el estado; la decisión la toma la arista condicional
def criticar(state: WriteState) -> dict:
    return {}

# 4. Arista condicional: ¿aprobamos, nos rendimos o seguimos revisando?
def route(state: WriteState) -> str:
    if state["score"] >= 8:
        return "aprobado"
    if state["revisiones"] >= 4:          # tope para no iterar infinito
        return "rendirse"
    return "revisar"                    # vuelve al escritor -> CICLO de reflexión

# 5. Construcción del grafo
g = StateGraph(WriteState)
g.add_node("escritor", escribir)
g.add_node("critico", criticar)

g.add_edge(START, "escritor")
g.add_edge("escritor", "critico")
g.add_conditional_edges(
    "critico",
    route,
    {"aprobado": END, "rendirse": END, "revisar": "escritor"},
)

app = g.compile()

# 6. Ejecución
final = app.invoke({"tema": "onboarding de usuarios", "borrador": "", "score": 0,
"revisiones": 0})

```

```
print(f"Final tras {final['revisiones']} revisiones | score {final['score']}")
print(final["borrador"])
```

Salida esperada:

[escritor] revisión 1 -> score 3 [escritor] revisión 2 -> score 6 [escritor] revisión 3 -> score 9 Final tras 3 revisiones | score 9 Borrador sobre 'onboarding de usuarios' (calidad 9/10)

Qué pasó, paso a paso:

1. El flujo es `escritor → crítico → (decisión)`. El crítico no cambia el estado; solo es el punto donde la arista condicional evalúa.
2. Tras cada vuelta, `route` mira el `score`: si llega a 8 termina (`aprobado`); si no, vuelve al escritor (`revisar`). **Ese retorno es el ciclo de reflexión.**
3. La condición `revisiones >= 4` es un **freno de seguridad**: sin él, un borrador que nunca llega a 8 generaría un loop infinito.
4. En 3 vueltas el score pasa $3 \rightarrow 6 \rightarrow 9$ y se aprueba.

A diferencia del primer ejemplo (ciclo de reintento ante un fallo), este es un ciclo de **mejora iterativa**: el sistema se autocritica hasta alcanzar un umbral de calidad. Es la base de los agentes "reflexivos".

8. Síntesis y Conexiones

En esta unidad hemos sentado las bases arquitectónicas de los sistemas de IA del futuro. En lugar de scripts lineales, ahora diseñamos **sistemas dinámicos**.

Puntos clave:

- **Estado**: La única fuente de verdad tipada que fluye por el sistema.
- **Nodos**: Funciones aisladas que transforman el estado.
- **Aristas**: Reglas de navegación que permiten ciclos y bifurcaciones.

En la siguiente unidad, daremos vida a estos nodos integrando **Tool Calling**, permitiendo que el estado no solo guarde texto, sino también los resultados de acciones ejecutadas en el mundo real. ¡Prepárate para conectar tu grafo con herramientas externas!

Tool Calling: Definición y Ejecución de Herramientas con LangChain

Aprende a definir, validar y ejecutar funciones externas (tools) que permiten a los LLMs interactuar con bases de datos y APIs externas de forma segura utilizando LangChain y Pydantic.

Tool Calling: Definición y Ejecución Segura

En el desarrollo de sistemas de IA, un LLM "desnudo" es como un cerebro brillante en una habitación sin ventanas: puede razonar, pero no puede interactuar con el mundo exterior. El **Tool Calling** (Invocación de Herramientas) es el mecanismo que le da "manos" a ese cerebro.

¿Qué es Tool Calling?

No es que el modelo ejecute código Python directamente. En su lugar, el modelo **genera una solicitud estructurada** (usualmente JSON) indicando qué función quiere ejecutar y con qué parámetros. Tu aplicación intercepta esa solicitud, ejecuta el código real, y le devuelve el resultado al modelo.

El "Contrato" de la Herramienta

Para que un modelo como GPT-4 o Claude 3.5 use una herramienta con éxito, necesita un contrato claro. Este contrato tiene cuatro pilares:

1. **Nombre:** Identificador único (ej. `get_stock_price`).
2. **Descripción:** Explicación semántica. El LLM decide si usar la herramienta basándose en esto.
3. **Esquema de Entrada (Input Schema):** Definición estricta de variables (tipos, obligatoriedad).
4. **Esquema de Salida:** Qué esperar de vuelta.

Definición Segura con Pydantic y LangChain

En entornos escalables, no podemos confiar en que el LLM envíe siempre los datos correctos. Usamos **Pydantic** para validar los tipos antes de que el código toque nuestra infraestructura.

CÓDIGO

```
from pydantic import BaseModel, Field
from langchain_core.tools import tool

class CalculatorInput(BaseModel):
    a: int = Field(description="Primer número")
    b: int = Field(description="Segundo día")

@tool(args_schema=CalculatorInput)
def multiply(a: int, b: int) -> int:
    """Multiplica dos números enteros."""
    return a * b
```

El Ciclo de Ejecución (Reason-Act Loop)

1. **Reasoning:** El LLM recibe el prompt y las definiciones de herramientas. Decide que necesita `multiply`.
2. **Tool Call:** El LLM responde con un objeto especial: `tool_calls: [{name: 'multiply', args: {a: 5, b: 2}}]`.
3. **Execution (Local):** Tu servidor detecta esta intención, valida los argumentos con Pydantic y ejecuta la función `multiply(5, 2)`.
4. **Observation:** El resultado (`10`) se envía de vuelta al LLM en un nuevo mensaje con el rol `tool`.
5. **Final Response:** El LLM procesa el `10` y responde al usuario: "El resultado es 10".

Errores Comunes y Seguridad

- **Confundir Invocación con Ejecución:** El LLM **no** ejecuta el código. Tu arquitectura debe garantizar que las herramientas tengan permisos limitados (Least Privilege). Nunca permitas que una herramienta ejecute SQL raw generado por el LLM sin validación.
- **Descripciones Ambiguas:** Si tu descripción es "Busca datos", el modelo fallará. Sé específico: "Busca precios de acciones en tiempo real para empresas del S&P 500".
- **Ignorar Errores:** Si la herramienta falla (ej. timeout de API), debes capturar el error y enviarlo como un string al LLM para que este pueda explicarle al usuario qué salió mal o intentar otra estrategia.

En la siguiente práctica, conectarás este concepto con el sistema RAG que construiste anteriormente, permitiendo que tu agente decida cuándo consultar la base de conocimientos.

Ciclos ReAct y Persistencia de Checkpoints en Agentes Autónomos

Implementa un agente autónomo con el ciclo ReAct y utiliza Checkpointers para persistir el estado de la ejecución y el historial de conversación.

Como arquitecto de IA, tu objetivo no es solo que el sistema responda, sino que sea **resiliente**. En sistemas complejos, un agente puede fallar a mitad de una ejecución de 10 pasos debido a un error de red o un timeout. Sin persistencia, perderías todo el razonamiento y los resultados parciales (y el dinero gastado en tokens).

1. El Ciclo ReAct (Reasoning + Acting)

El patrón ReAct es el estándar de oro para agentes autónomos. A diferencia de una cadena lineal, ReAct permite al LLM:

1. **Pensar (Reason):** Analizar la solicitud del usuario y decidir qué necesita.
2. **Actuar (Act):** Seleccionar una herramienta y generar los parámetros de llamada.
3. **Observar (Observe):** Recibir la salida de la herramienta y re-evaluar el estado.

Este proceso es **cíclico**. El LLM puede decidir que necesita usar tres herramientas diferentes antes de dar una respuesta final. LangGraph facilita esto permitiendo que una arista (edge) vuelva al mismo nodo o a un nodo anterior, creando un historial de mensajes que crece con cada "observación".

2. Checkpoints: La Caja Negra de tu Agente

En LangGraph, un **Checkpoint** es un mecanismo de almacenamiento que guarda una "fotografía" del estado del grafo después de cada nodo ejecutado.

- **Para qué sirve:** Si el proceso se detiene, puedes reanudarlo simplemente pasando el mismo `thread_id`. El sistema buscará en la base de datos (SQLite, Redis o Memoria) y cargará el último estado conocido.

- **Time Travel:** Los checkpoints permiten no solo reanudar, sino "volver atrás" en el tiempo (debugging) o requerir aprobación humana antes de continuar con un nodo crítico (Human-in-the-loop).

Enunciado del Problema

Vas a construir un agente climatológico capaz de razonar si necesita herramientas y que no pierda la memoria incluso si interrumpimos la ejecución del script.

Objetivo:

1. Implementar un ciclo ReAct completo.
2. Configurar persistencia en memoria.
3. Demostrar que el agente recuerda la ubicación mencionada en el primer mensaje al recibir un segundo mensaje, gracias al `thread_id`.

Restricciones:

- Usa `Annotated[list[BaseMessage], add_messages]` en tu estado para asegurar que los mensajes no se sobrescriban, sino que se concatenen.
- El grafo debe terminar solo cuando el LLM decida que no hay más herramientas que llamar.
- No olvides que el nodo de herramientas (`ToolNode`) debe mapear sus resultados de vuelta al agente.

Agente de razonamiento cíclico con memoria persistente

Especificación de la pre-entrega para el diseño de un agente autónomo cíclico con LangGraph, integrando persistencia y ejecución de herramientas externas.

Criterios de Aceptación

Para que este checkpoint se considere aprobado, el entregable debe cumplir con lo siguiente:

- **Autonomía:** El agente debe ser capaz de determinar por sí mismo cuándo llamar a una herramienta basándose en el prompt del usuario (sin rutas manuales `if/else`).
- **Ciclo de Retorno:** Si una herramienta devuelve un error o información incompleta, el agente debe ser capaz de realizar un segundo intento o pedir aclaraciones.
- **Resiliencia de Estado:** Al proporcionar un `thread_id`, el agente debe ser capaz de recordar interacciones previas dentro de una misma sesión de razonamiento.
- **Código Limpio:** Uso de Python 3.12, tipado estático (Type Hints) y gestión asíncrona (`asyncio`).

Guía de Implementación Sugerida

Fase 1: El Contrato de Herramientas

Define tus funciones utilizando el decorador `@tool` de LangChain. Asegúrate de incluir docstrings extremadamente descriptivos; recuerda que el LLM decide qué herramienta usar basándose únicamente en esa descripción.

Fase 2: Definición del Estado y el Grafo

Crea el esquema de tu estado. En LangGraph, el estado es inmutable y se actualiza mediante reducers (usualmente `operator.add` para la lista de mensajes). Configura el `StateGraph` conectando el nodo del modelo con el nodo de ejecución de herramientas mediante una arista condicional (`tools_condition`).

Fase 3: Persistencia

Configura un `Checkpoint`. Esto es lo que permite que el agente sea "arquitectónicamente escalable". Sin persistencia, tu agente es efímero; con ella, es capaz de manejar flujos de trabajo largos que requieren intervención humana o esperar procesos externos.

Errores Comunes a Evitar

- **Descripciones Vagaces:** Si el agente no usa la herramienta que esperas, el error suele estar en el docstring de la función, no en la lógica del grafo.

- **Bucles Infinitos:** No establecer un límite de recursión (`recursion_limit`) al invocar el grafo. Define siempre un techo (ej. 10 pasos) para evitar costos inesperados en la API.
- **Estado Sucio:** Olvidar que el estado se acumula. Asegúrate de limpiar o resumir mensajes si el contexto se vuelve demasiado grande.

Qué entregás y en qué formato

- **Tipo:** Código — un repositorio público de GitHub.
- **Artefacto concreto:** repo con el `StateGraph` , al menos una herramienta propia, persistencia (`SqliteSaver`), la traza ReAct como log o `.json` , y un `README.md` .
- **Qué NO hace falta:** no entregás documento aparte; la traza del razonamiento se entrega como log/JSON dentro del repo.

Pasos para la entrega

1. Inicializa un proyecto Python 3.12+ con Poetry o venv.
2. Define un `StateGraph` que herede de `MessagesState` .
3. Crea al menos una herramienta personalizada que simule una operación de base de datos o búsqueda técnica.
4. Configura un LLM (OpenAI o Anthropic) vinculado a las herramientas (`llm.bind_tools()`).
5. Implementa la lógica de persistencia usando `SqliteSaver` (para desarrollo local).
6. Realiza una prueba de ejecución donde el agente deba llamar a la herramienta al menos dos veces para llegar a la conclusión (razonamiento multi-paso).
7. Sube el código a un repositorio público y asegúrate de no incluir las API Keys (usa variables de entorno).

Ejemplo de la traza esperada (razonamiento cíclico)

Tu agente debe mostrar un ciclo ReAct como este (incluílo como log o `.json` en el repo):

CÓDIGO

```

Usuario: "¿Cuántos pedidos tuvo el cliente 102 y cuál fue el total?"
→ El agente decide usar la herramienta: buscar_pedidos(cliente_id=102)
→ La herramienta devuelve: { "pedidos": 3, "total": 14500 }
→ El agente razona: ya tiene los datos → responde.
Respuesta: "El cliente 102 tuvo 3 pedidos por un total de $14.500."

(Con el mismo thread_id, si después preguntás "¿y el último?", el agente recuerda el contexto.)

```

Checklist de entrega

- Repo público (sin API keys; variables de entorno) con README de cómo levantar el entorno.
- `StateGraph` (hereda de `MessagesState`) con nodo de modelo + nodo de herramientas y arista condicional (`tools_condition`).

- Al menos 1 herramienta con `@tool` y docstring descriptivo.
- Persistencia con `SqliteSaver` + `thread_id` (recuerda la sesión).
- Prueba con razonamiento multi-paso (la herramienta se invoca ≥ 2 veces) y `recursion_limit` definido.
- Ejemplo de traza de ejecución (.json o log) incluido.

Entregable

Formato: repositorio

Repositorio de GitHub que contenga el código del agente en un entorno asíncrono, incluyendo la definición del grafo, las herramientas y la configuración del checkpointer. Debe incluir un archivo README explicando cómo levantar el entorno y un ejemplo de traza de ejecución (.json o log).

Objetivos de aprendizaje

- Diseñar e implementar un flujo de trabajo cíclico (ReAct) utilizando LangGraph.
- Configurar la persistencia de estado mediante Checkpointers para garantizar la continuidad de la ejecución.
- Definir herramientas técnicas robustas que los LLMs puedan invocar de forma autónoma mediante esquemas estructurados.
- Gestionar estados complejos en sistemas de IA asíncronos siguiendo principios de ingeniería de software.

Rúbrica de evaluación

Esta rúbrica evalúa la capacidad del estudiante para diseñar e implementar un agente autónomo utilizando LangGraph, integrando ciclos de razonamiento ReAct, herramientas personalizadas y persistencia de estado para sistemas de IA escalables profesionales.

Criterio	Descripción	Peso
Arquitectura del Grafo y Gestión de Estado	Evaluación de la correcta implementación del StateGraph y el uso de MessagesState para mantener la coherencia del diálogo.	30%
Integración de Herramientas y Razonamiento Multi-paso	Capacidad del agente para invocar herramientas externas y procesar información en múltiples iteraciones.	35%
Persistencia con SqliteSaver	Implementación de mecanismos de memoria para permitir la continuidad de las conversaciones.	20%
Calidad del Repositorio y Entorno Profesional	Uso de estándares de la industria en la organización del código, gestión de dependencias y seguridad de credenciales.	15%

Total: 100 pts | Aprobación: 70 pts

Sistemas multi-agente: colaboración y especialización

Diseñar sistemas complejos donde múltiples agentes autónomos colaboran y se especializan para resolver tareas de alto nivel.

Topologías Multi-Agente: Colaboración vs Jerarquía

Explora las estructuras fundamentales de organización en sistemas multi-agente, analizando cuándo centralizar el control mediante jerarquías o descentralizarlo mediante colaboración para maximizar la eficiencia y modularidad.

Imagina que estás diseñando un sistema de software para gestionar un hospital inteligente. Tienes diferentes necesidades: diagnosticar enfermedades raras, gestionar el inventario de medicamentos, optimizar las rondas de las enfermeras y responder a emergencias en tiempo real. ¿Deberías crear un único programa "todopoderoso" que intente resolverlo todo? ¿O sería mejor tener especialistas que hablen entre sí?

Si optas por especialistas, acabas de entrar en el mundo de los **Sistemas Multi-Agente (MAS)**. Pero aquí surge el verdadero reto de ingeniería: ¿cómo deben relacionarse estos agentes? ¿Necesitan un "director de orquesta" que les asigne tareas (Jerarquía), o pueden negociar entre ellos como un equipo de expertos en una sala de juntas (Colaboración)?

En esta unidad, exploraremos las topologías que definen el flujo de control y comunicación en arquitecturas de IA escalables, sentando las bases para construir sistemas que no solo "funcionen", sino que sean mantenibles y resilientes.

1. El Salto de la Cadena al Sistema

Hasta ahora, en los módulos anteriores, hemos trabajado principalmente con **cadena lineales** o **ciclos ReAct dirigidos**. Un usuario pregunta, un agente razona, usa una herramienta y responde.

Sin embargo, cuando el problema es demasiado grande para un solo prompt o un solo bucle de razonamiento (por ejemplo, escribir un libro entero, auditar un sistema financiero o simular una cadena de suministro), la complejidad cognitiva desborda a un único agente. Aquí es donde la **topología** se vuelve crítica.

La **topología multi-agente** es la estructura de red que define quién tiene la autoridad, cómo se distribuye el estado y qué canales de comunicación existen entre los agentes. No es solo un diagrama; determina cómo escalará tu sistema bajo carga y qué tan fácil será depurarlo.

2. Topología Jerárquica: El Modelo del Supervisor

La arquitectura jerárquica es, quizás, la más intuitiva para un desarrollador de software. Se basa en una estructura de mando clara donde un agente de nivel superior (Supervisor o Coordinador) gestiona a un grupo de agentes especialistas (Workers).

¿Cómo funciona?

1. **Delegación:** El usuario envía una petición al Supervisor.
2. **Descomposición:** El Supervisor analiza la tarea y la divide en subtareas más pequeñas.
3. **Asignación:** El Supervisor "llama" a los agentes específicos (ej. Agente de Búsqueda, Agente de Redacción, Agente de Validación).
4. **Agregación:** Los especialistas devuelven sus resultados al Supervisor, quien sintetiza la respuesta final.

Ejemplo Real: Agencia de Viajes Autónoma

Imagina un sistema para planificar viajes complejos:

- **Agente Supervisor (El Planner):** Recibe "Quiero un viaje a Japón de 10 días". Decide que necesita un vuelo, hoteles y un itinerario.
- **Agente Especialista A (SkyScanner Expert):** Solo sabe buscar vuelos y manejar fechas.
- **Agente Especialista B (Booking Specialist):** Solo sabe de alojamientos y disponibilidad.
- **Agente Especialista C (Cultural Guide):** Genera descripciones de lugares basadas en intereses.

El Planner coordina. Si el Agente de Vuelos falla, el Planner decide si intenta de nuevo o busca una alternativa, sin que el Agente de Hoteles se entere siquiera del problema.

Ventajas y Desventajas

Ventaja	Explicación
Control Centralizado	Es fácil predecir el flujo y auditar las decisiones en un solo punto (el supervisor).
Modularidad	Puedes cambiar el Agente de Vuelos por uno mejor sin tocar el resto del sistema.
Reducción de Ruido	Los especialistas no reciben información irrelevante; solo su tarea específica.

El Gran Riesgo: El supervisor es un **cuello de botella**. Si el LLM que actúa como supervisor se confunde o falla, todo el sistema queda paralizado. Además, a medida que añades más especialistas, la "carga cognitiva" del supervisor aumenta, lo que puede requerir modelos más costosos (como GPT-4o o Claude 3.5 Sonnet) solo para la coordinación.

3. Topología Cooperativa: El Modelo de Colaboración (Peer-to-Peer)

En una topología cooperativa o descentralizada, no hay un jefe. Los agentes son "pares" (peers) que se comunican entre sí para resolver un problema. La coordinación surge de un **estado compartido** o de un protocolo de comunicación acordado.

¿Cómo funciona?

En lugar de esperar órdenes, los agentes observan el estado del sistema y actúan cuando ven que pueden aportar algo.

- **Pizarrón (Blackboard):** Un patrón común donde todos los agentes escriben en un espacio compartido. Si el "Agente Traductor" ve un texto en inglés en el pizarrón, lo traduce automáticamente al español sin que nadie se lo pida explícitamente.
- **Coreografía:** Cada agente sabe qué hacer cuando recibe un mensaje de otro, creando una reacción en cadena.

Ejemplo Real: Desarrollo de Software Colaborativo

Imagina tres agentes trabajando en un repositorio de código:

1. **Agente Arquitecto:** Escribe la especificación de una función en el estado compartido.
2. **Agente Programador:** Al detectar una nueva especificación, escribe el código.
3. **Agente Tester:** Al detectar código nuevo, escribe y ejecuta pruebas.
4. **Agente Programador (de nuevo):** Si el Tester publica errores, el Programador los corrige automáticamente.

Aquí no hay un supervisor diciendo "ahora tú, ahora tú". El flujo es reactivo y dinámico.

Ventajas y Desventajas

Ventaja	Explicación
Resiliencia	No hay un punto único de fallo. Si un agente se cae, otros pueden seguir trabajando en tareas paralelas.
Escalabilidad Orgánica	Es más fácil añadir agentes que "escuchen" eventos sin reconfigurar un orquestador central.
Adaptabilidad	Ideal para entornos donde el orden de las tareas no es predecible.

El Gran Riesgo: El caos emergente. Sin una supervisión estricta, los agentes pueden entrar en bucles infinitos (el programador cambia algo, el tester falla, el programador vuelve a cambiar, y así para siempre) o producir resultados inconsistentes. La depuración (debugging) se vuelve significativamente más compleja al no haber un rastro lineal de ejecución.

4. Comparativa Técnica para el Arquitecto de IA

Como desarrollador Python, tu elección de topología suele depender de la **impureza del problema**.

¿Cuándo elegir Jerarquía?

- Cuando tienes un **workflow determinista** o semi-determinista.
- Cuando la seguridad es crítica y necesitas un "validador" final (el supervisor) que revise todo antes de mostrarlo al usuario.

- Cuando usas modelos de diferentes tamaños (ej. un Supervisor "inteligente" GPT-4o controlando a muchos Workers "rápidos" y baratos GPT-4o-mini).

¿Cuándo elegir Colaboración?

- Cuando el problema es **explorativo** (ej. investigación de mercado donde no sabes qué vas a encontrar).
- Cuando los agentes necesitan trabajar en paralelo masivo sobre un mismo conjunto de datos.
- En sistemas de tiempo real donde la latencia de un supervisor centralizado sería inaceptable.

5. Errores Comunes y Mejores Prácticas

Error 1: El "Supervisor Todólogo"

Un error frecuente es diseñar un supervisor que intenta hacer parte del trabajo de los agentes. Esto infla el prompt del supervisor, gasta tokens innecesarios y aumenta la probabilidad de alucinaciones.

- **Solución:** El supervisor debe ser un **director de tráfico**, no un ejecutor. Su prompt debe enfocarse exclusivamente en lógica de división de tareas y evaluación de resultados.

Error 2: Falta de Contratos de Interfaz

En topologías cooperativas, los desarrolladores a menudo olvidan definir *exactamente* cómo debe verse el dato que un agente pasa al siguiente.

- **Solución:** Utiliza **Pydantic** para definir esquemas robustos. Si el Agente A produce un JSON, el Agente B debe saber exactamente qué llaves esperar. En LangGraph, esto se gestiona mediante el `State` compartido.

Error 3: El Bucle Infinito de Corrección

En arquitecturas tipo Critic-Refiner (una variante de colaboración), es común que el Crítico siempre encuentre algo mal y el Refinador siempre intente arreglarlo, gastando presupuesto de API sin fin.

- **Solución:** Implementa un **contador de intentos** o una condición de parada basada en una métrica de calidad (ej. "detener si la puntuación de calidad es > 0.8 o si llevamos 3 iteraciones").

6. Síntesis: El camino hacia sistemas complejos

Diseñar sistemas multi-agente es elevarse un nivel por encima del código tradicional. Ya no solo programas instrucciones; estás diseñando una **organización digital**.

- La **Jerarquía** te da orden, seguridad y trazabilidad, pero introduce cuellos de botella.
- La **Colaboración** te da flexibilidad, velocidad y resiliencia, pero a costa de una mayor complejidad de depuración.

En las próximas unidades, veremos cómo implementar el **Patrón Supervisor** en código usando LangGraph, donde aprenderás a gestionar el estado entre estos agentes y a evitar que tu "director de orquesta" se sature. Recuerda: la mejor topología no es la más compleja, sino la más simple que logre resolver el problema de negocio con la menor latencia y costo posibles.

Conexión con lo que viene: Ahora que entiendes la teoría de cómo se organizan los agentes, en la siguiente unidad práctica meteremos las manos en la masa para construir un **Supervisor Agent**. Veremos cómo LangGraph facilita la delegación dinámica y cómo podemos usar nodos para representar a estos especialistas que acabamos de describir.

El Patrón Supervisor para Delegación Dinámica

Aprende a implementar el patrón Supervisor para coordinar múltiples agentes especialistas de forma dinámica utilizando LangGraph.

El Patrón Supervisor: Delegación Dinámica

En sistemas multi-agente, la complejidad crece exponencialmente con cada nuevo "agente" que añadimos. Si conectamos todos con todos (una topología de red), el ruido y la falta de control hacen que el sistema sea impredecible. Aquí es donde entra el **Patrón Supervisor**.

¿Qué es el Patrón Supervisor?

Imagina una redacción de noticias: tienes reporteros (investigadores) y redactores. No se mandan correos entre ellos de forma caótica; hay un **Editor Jefe** (el Supervisor) que recibe la noticia, decide quién debe investigar, revisa el borrador y finalmente aprueba la publicación.

En **LangGraph**, el Supervisor es un nodo central que:

1. **Analiza el estado actual:** Lee el historial de mensajes para entender qué se ha hecho.
2. **Toma una decisión de ruteo:** Decide qué especialista es el más apto para el siguiente paso.
3. **Valida el progreso:** Actúa como un "control de calidad", pudiendo pedir a un agente que repita su tarea si no es satisfactoria.

¿Por qué usar este patrón en lugar de una cadena simple?

- **Especialización de Modelos:** Puedes usar un modelo costoso (GPT-4o) para el Supervisor y modelos ligeros (Llama-3-8B) para los agentes especialistas.
- **Escalabilidad:** Añadir una nueva funcionalidad es tan sencillo como crear un nodo y registrarlo en las opciones del Supervisor.
- **Recuperación de Errores:** Si un agente falla, el Supervisor puede decidir reintentar o cambiar de estrategia.

Implementación en LangGraph

Para implementar este patrón, dependemos de las **Aristas Condicionales**. El supervisor no tiene una ruta fija; su salida determina el flujo dinámicamente.

Un error común es intentar que el Supervisor haga el trabajo de los especialistas. **Recuerda:** El Supervisor coordina, el Especialista ejecuta. Si tu Supervisor está escribiendo código, tu arquitectura está perdiendo la ventaja de la especialización.

En esta unidad, construiremos un orquestador que delegará tareas de análisis y escritura de forma inteligente.

Estado Compartido y Comunicación Asíncrona

Aprende a gestionar el estado compartido y la comunicación no bloqueante entre agentes utilizando Asyncio, LangGraph y Pydantic para asegurar la integridad de los datos.

En sistemas multi-agente, la comunicación no es simplemente "enviar un mensaje". Se trata de gestionar la **coherencia del conocimiento** a medida que este evoluciona. Cuando trabajamos con agentes especializados, el mayor desafío no es que el LLM genere texto, sino que todos los componentes del sistema operen sobre una versión válida y actualizada de la realidad.

Por qué Asyncio es vital en Arquitecturas de IA

Los LLMs son intrínsecamente lentos. Una llamada a GPT-4 o Claude 3.5 puede tardar varios segundos. Si tienes 5 agentes trabajando de forma síncrona, el tiempo de respuesta total será la suma de todos los retardos.

Utilizar `asyncio` permite que mientras un agente espera la respuesta de una API, el orquestador pueda:

1. Procesar resultados de otros agentes.
2. Realizar escrituras en bases de datos vectoriales.
3. Mantener la interfaz de usuario receptiva (streaming).

El Estado Compartido: De Diccionarios a Esquemas Validados

En unidades anteriores usamos diccionarios simples. Sin embargo, en un sistema de grado industrial, necesitamos **contratos**.

Un error común es permitir que cualquier agente modifique cualquier parte del estado. Esto conduce a **condiciones de carrera** (race conditions) donde el Agente B sobrescribe lo que el Agente A acaba de calcular basándose en datos obsoletos.

Estrategia Recomendada:

- **Aislamiento de Escritura:** Cada agente es dueño de una "parcela" del estado. Por ejemplo, el agente `researcher` solo escribe en `state["research_results"]`.
- **Validación con Pydantic:** Antes de "inyectar" datos en el estado compartido, los validamos contra un modelo de Pydantic. Si el LLM alucina un formato de fecha incorrecto o un score fuera de rango, el error se captura inmediatamente.

Consistencia Eventual

En sistemas asíncronos, aceptamos que por unos milisegundos, algunos agentes podrían ver una versión del estado ligeramente antigua. Esto se llama **Consistencia Eventual**. En LangGraph,

gestionamos esto mediante el flujo de nodos: un nodo solo se ejecuta cuando tiene la garantía de que sus dependencias (aristas de entrada) han completado su escritura.

Errores Comunes

1. **Shared Mutable State:** Intentar usar una lista global de Python y hacer `.append()` desde múltiples funciones `async`. Esto fallará en entornos distribuidos o bajo alta concurrencia. Usa siempre el mecanismo de retorno de estados del grafo.
2. **No manejar excepciones en Nodos:** Si un nodo asíncrono falla y no se captura, puede dejar el estado del sistema en un limbo ("bloqueado"). Siempre valida los outputs del LLM antes de retornarlos al grafo.

Orquestador multi-agente especializado

Briefing de la pre-entrega final del módulo, enfocada en la construcción de un sistema jerárquico de supervisor y agentes especializados utilizando LangGraph.

¿Qué debes construir?

Debes entregar un prototipo funcional de un **Orquestador Multi-Agente de Análisis e Investigación**. El sistema debe ser capaz de procesar una solicitud de usuario que requiera al menos dos dominios de especialización distintos y una fase de síntesis final.

Requerimientos Técnicos:

1. **Topología Jerárquica:** Un nodo `Supervisor` que actúa como router inteligente y controlador de flujo.
2. **Mínimo de 2 Agentes Especialistas:**
 - **Agente de Búsqueda/Investigación:** Con herramientas para consultar fuentes externas (puedes usar Tavily o una búsqueda simulada sobre tu Vector DB de pre-entregas anteriores).
 - **Agente de Análisis/Cómputo:** Especializado en procesar los datos obtenidos (análisis de sentimiento, cálculos matemáticos o validación de esquemas).
3. **Estado Compartido Estructurado:** Un esquema de `State` en LangGraph que permita rastrear qué agente ha contribuido con qué información, evitando la pérdida de contexto en la comunicación asíncrona.
4. **Flujo de Supervisión:** El supervisor debe decidir si la tarea está completa o si algún especialista debe refinar su output antes de dar la respuesta final.

Pasos Sugeridos

1. **Define tu Estado:** Diseña una clase `TypedDict` que herede de `MessagesState`. Piensa si necesitas campos extra como `next_agent` o `task_completed`.
2. **Crea los Agentes Especialistas:** Define funciones que utilicen `create_react_agent` de LangGraph o prompts específicos para cada rol. Asegúrate de que cada uno tenga herramientas acotadas.
3. **Implementa el Supervisor:** Este es el cerebro. Su prompt debe ser claro: "Dada la conversación actual, ¿quién debe intervenir ahora o es momento de finalizar?". Debe mapear sus respuestas a los nombres de los nodos del grafo.
4. **Construye el Grafo:** Une los puntos. Usa `add_node` para cada agente y para el supervisor. Define las `Conditional Edges` que conecten al supervisor con los especialistas.

5. **Prueba la Interacción:** Lanza una consulta que obligue al supervisor a enviar la tarea al Investigador, recibir el dato, enviarlo al Analista y finalmente cerrar la conversación.

Errores Comunes a Evitar

- **El "Supervisor Infinito":** No definir una condición de parada clara, haciendo que el supervisor y los agentes entren en un bucle de correcciones eternas. Tip: Implementa un contador de pasos o un criterio de "Suficiencia" estricto.
- **Contaminación de Contexto:** Pasar todo el historial a todos los agentes en cada turno. Para sistemas escalables, a veces es mejor que el especialista solo reciba la instrucción específica y el contexto necesario, no toda la metadata del sistema.

Qué entregás y en qué formato

- **Tipo:** Código — un repositorio de GitHub.
- **Artefacto concreto:** repo con `state.py`, la carpeta `agents/` (investigación + análisis), el grafo con nodo Supervisor, y un `README.md` con el diagrama Mermaid del grafo. Sumá un video corto o notebook que demuestre el flujo de delegación.
- **Qué NO hace falta:** el video/notebook es una demo del flujo, no un informe escrito.

Pasos para la entrega

1. Estructura del Repositorio:

- Crea un archivo `state.py` para definir el esquema de datos compartido.
- Crea un directorio `agents/` con los especialistas (ej. `research_agent.py`, `analyst_agent.py`).
- Define el grafo principal en `main.py` o `graph.py`.

2. Implementación del Grafo:

- Utiliza `StateGraph` de `LangGraph`.
- Define al menos un nodo de "Supervisor" que decida dinámicamente el flujo usando `Literal` en el retorno para las aristas condicionales.

3. Herramientas (Tools):

- Los agentes deben tener al menos una herramienta funcional (puedes usar `TavilySearchResults` o una función propia que consulte una base de datos).

4. Validación:

- Implementa un nodo de `Validation` o asegura que el `Supervisor` tenga una rúbrica personalizada en su prompt para validar los resultados de los especialistas antes de dar el `END`.

5. Documentación:

- Tu README debe incluir un diagrama del grafo (puedes generarlo con `graph.get_graph().draw_mermaid_png()`).
- Explica brevemente por qué elegiste esa topología y cómo manejas los posibles conflictos entre agentes.

Entregable

Formato: repositorio

Repositorio de GitHub con el código del orquestador, un archivo README explicando la topología elegida y un video corto o notebook demostrando el flujo de delegación.

Objetivos de aprendizaje

- Diseñar arquitecturas de supervisor agent para la coordinación de tareas complejas.
- Implementar comunicación asíncrona y estado compartido entre agentes mediante LangGraph.
- Configurar lógica de validación de outputs en flujos multi-paso para garantizar la calidad de la respuesta.
- Aplicar principios de ingeniería como tipado estático y manejo de errores asíncronos en sistemas multi-agente.

Rúbrica de evaluación

Evaluación de la capacidad para diseñar y ejecutar arquitecturas multi-agente jerárquicas utilizando LangGraph, con énfasis en la gestión de estado, delegación dinámica y validación de resultados.

Criterio	Descripción	Peso
Documentación y Visualización	Claridad en la explicación de la topología y representación visual del grafo en el README.	20%
Arquitectura y Flujo del Grafo (Supervisor)	Evaluación del diseño del StateGraph y la implementación del nodo Supervisor para la delegación dinámica.	35%
Especialización de Agentes y Herramientas	Capacidad de los agentes para ejecutar tareas específicas mediante el uso de herramientas funcionales.	25%
Gestión de Estado y Validación	Uso de state.py para la coherencia de datos y presencia de mecanismos de validación de resultados.	20%

Total: 100 pts | Aprobación: 70 pts

Producción y robustez: observabilidad, costos y despliegue

Asegurar la robustez del sistema mediante observabilidad detallada, control de costos y despliegue asíncrono bajo estándares de ingeniería.

Observabilidad y Trazado: Arize Phoenix y LangSmith

Aprende a implementar sistemas de observabilidad y trazado avanzado utilizando Arize Phoenix y LangSmith para monitorear latencia, costos y calidad en aplicaciones de IA de producción.

Bienvenido al Módulo 7. Hasta este punto, has construido sistemas de IA capaces de razonar, recuperar información y colaborar en arquitecturas multi-agente. Sin embargo, en producción, **"funciona en mi computadora"** es una frase peligrosa: los sistemas basados en LLMs son probabilísticos y "opacos", y pueden fallar sin lanzar ningún error (lo vemos en detalle en la sección 1).

En esta unidad, aprenderás a implementar **observabilidad de nivel empresarial** utilizando dos de las herramientas más potentes del ecosistema: **Arize Phoenix** y **LangSmith**. Pasaremos de "adivinar" qué sucede en tus cadenas a tener una visibilidad total sobre cada span, cada token y cada milisegundo.

Lo esencial en 3 líneas

1. **Qué construís:** instrumentás tu app de IA con observabilidad (trazas y spans) usando Arize Phoenix o LangSmith.
2. **Por qué:** los LLMs fallan "en silencio" (responden 200 OK pero alucinan o tardan de más); sin trazas, no sabés qué pasó.
3. **Con las secciones 1-3 ya tenés el núcleo;** las secciones 4-7 son profundización opcional.

1. El Problema: El "Éxito Silencioso" y la Opacidad

En el desarrollo de software tradicional, un error suele ser binario: el código rompe o funciona. En la ingeniería de IA, enfrentamos el fenómeno del **"éxito silencioso"**:

- El usuario recibe una respuesta bien formateada.
- El status code de la API es 200 OK.
- **Pero:** El agente decidió ignorar las herramientas, usó un contexto recuperado irrelevante o alucinó una cifra financiera crítica.

Sin observabilidad, estás volando a ciegas. La observabilidad no es solo "loggear" texto; es la capacidad de reconstruir el estado interno del sistema a partir de sus datos externos (trazas, métricas y evaluaciones).

2. Conceptos Fundamentales: Trazas, Spans y Runs

Para dominar herramientas como Arize Phoenix o LangSmith, primero debemos entender la anatomía de una ejecución de IA:

1. **Trace (Trazo):** Es el viaje completo de una solicitud, desde que el usuario envía un mensaje hasta que recibe la respuesta final. Por ejemplo, una traza de un agente multi-paso incluiría todas las iteraciones de razonamiento.
2. **Span (Lapso):** Es una unidad individual de trabajo dentro de una traza. Un span puede ser una llamada a una base de datos vectorial, una invocación al modelo (LLM Call), o la ejecución de una herramienta (Tool Call). Los spans son jerárquicos: una traza es un árbol de spans.
3. **Run (Ejecución):** En el contexto de LangSmith, se refiere a la instancia de ejecución de un `Runnable` de LangChain.

¿Por qué separar en Spans?

Imagina que tu sistema tarda 10 segundos en responder. Si solo tienes un log del inicio y el final, no sabes si el problema fue la latencia de OpenAI, una búsqueda lenta en Pinecone o un bucle infinito en tu lógica de LangGraph. Los spans te permiten aislar el cuello de botella instantáneamente.

3. Arize Phoenix vs. LangSmith: ¿Cuál elegir?

Aunque ambas herramientas resuelven problemas similares, sus filosofías y casos de uso difieren.

Arize Phoenix: El Estándar Abierto y Local-First

Phoenix es una plataforma de observabilidad de código abierto que se integra profundamente con el estándar **OpenInference** (una extensión de OpenTelemetry para IA).

- **Ventaja Principal:** Es *framework-agnostic*. No importa si usas LangChain, LlamaIndex o el SDK puro de OpenAI; si emites datos compatibles con OpenTelemetry, Phoenix los visualiza.
- **Privacidad:** Puedes correrlo localmente en un contenedor Docker o directamente en un entorno de Notebook. Los datos no tienen que salir de tu infraestructura para ser analizados.
- **Enfoque:** Análisis de *embeddings*, detección de *drift* (deriva de datos) y evaluaciones de RAG (groundedness).

LangSmith: La Experiencia Integrada de LangChain

Desarrollado por el equipo detrás de LangChain, es la opción natural si tu stack está construido sobre LCEL (LangChain Expression Language) o LangGraph.

- **Ventaja Principal:** Integración "plug-and-play". Con solo configurar dos variables de entorno, cada paso de tu aplicación LangChain se registra automáticamente con una granularidad asombrosa.
- **Debugging:** Permite editar y re-ejecutar prompts directamente desde la interfaz para ver cómo cambia el output sin necesidad de reiniciar tu servidor.
- **Datasets:** Muy fuerte en la creación de conjuntos de datos de prueba a partir de trazas de producción para realizar pruebas de regresión.

Profundización (opcional)

Con las secciones 1-3 ya tenés el núcleo del tema. Lo que sigue (secciones 4 a 7) amplía la implementación técnica y los casos avanzados; leelo cuando lo necesites.

4. Implementación y Trazado Avanzado

El Enfoque OpenTelemetry en Arize Phoenix

Como vimos en la sección 3, Phoenix se apoya en OpenInference/OpenTelemetry (por eso es *framework-agnostic* y portable). Lo que importa acá es el flujo de datos al instrumentar una aplicación:

1. **Instrumentation:** Librerías que "envuelven" tus llamadas a OpenAI/Anthropic/LangChain.
2. **Exporter:** Envía los datos capturados al colector de Phoenix.
3. **Frontend:** Visualización de trazas en `localhost:6006`.

Monitoreo de Latencia y Costos

Uno de los mayores errores de principiante es monitorear solo la latencia promedio. En producción, lo que importa son los **percentiles (p95, p99)**.

- **p50:** La mediana del tiempo de respuesta.
- **p95:** El tiempo en el que el 5% de tus usuarios están experimentando retrasos. Si tu p95 es de 45 segundos, tienes un problema grave de experiencia de usuario, incluso si tu promedio es de 3 segundos.

Tanto Phoenix como LangSmith calculan automáticamente el costo basado en los **tokens de entrada y salida** informados en los metadatos de la respuesta del modelo, permitiéndote detectar qué agentes o usuarios están consumiendo el presupuesto de forma desproporcionada.

5. Detección de Alucinaciones y Evaluadores (Evals)

El monitoreo de software tradicional usa "checks" booleanos. En IA, usamos **Evaluadores** (o *LLM-as-a-Judge*).

La Tríada de Evaluación en RAG:

Cuando implementamos RAG (como vimos en el Módulo 3 y 4), la observabilidad nos permite medir tres métricas críticas para detectar alucinaciones:

1. **Faithfulness (Fidelidad):** ¿La respuesta del LLM se deriva exclusivamente del contexto recuperado?
Si el LLM usa conocimiento interno para responder algo que no estaba en los documentos, se considera una alucinación (incluso si la respuesta es correcta en el mundo real), porque rompe el control del sistema.
2. **Answer Relevance:** ¿La respuesta realmente aborda la pregunta del usuario?
3. **Context Precision:** ¿Los documentos recuperados de la Vector DB eran realmente útiles para responder la pregunta?

Ejemplo

Ejemplo de Error Común: Muchos desarrolladores intentan solucionar una mala respuesta cambiando el prompt, cuando la traza en Phoenix muestra que el problema es de **Context Recall**: el retriever no trajo la información necesaria de Pinecone. Sin ver el span de recuperación, estarías optimizando el componente equivocado.

6. Errores Comunes y Mejores Prácticas

Error 1: Instrumentar demasiado tarde

No esperes a llegar al despliegue para activar LangSmith o Phoenix. La observabilidad es una herramienta de **desarrollo**. Ver las trazas mientras programas te ayuda a entender cómo fluye el estado en tus grafos de LangGraph de manera visual, lo cual es mucho más eficiente que llenar el código de `print()`.

Error 2: Ignorar las versiones de los Prompts

Una alucinación puede ser causada por un cambio de una sola palabra en un System Prompt.

- **Mejor Práctica:** Siempre loguea el `prompt_template_version` como un tag en tus trazas. LangSmith permite gestionar esto nativamente con su "Prompt Hub".

Error 3: Confiar ciegamente en los auto-evaluadores

Los evaluadores basados en LLM (como los de Phoenix) pueden tener sus propios sesgos. No reemplazan las pruebas unitarias tradicionales para lógica determinística ni la revisión humana para casos críticos. Úsalos como señales de alerta temprana, no como verdades absolutas.

7. Síntesis y Conexión

Hemos pasado de ver la IA como una "caja negra" a entenderla como un flujo de datos estructurado y medible.

- **Tracing** nos da la historia de lo que pasó.
- **Spans** nos dicen dónde falló o se ralentizó.
- **Evals** nos dicen si el resultado es confiable.

En la próxima unidad, combinaremos esta visibilidad con la infraestructura necesaria para escalar: aprenderemos cómo **FastAPI y Redis** permiten que este sistema trazable responda a miles de usuarios de forma asíncrona sin bloquear el hilo principal de ejecución. La observabilidad que configures hoy será tu salvavidas cuando el sistema crezca mañana.

Escalabilidad Asíncrona con FastAPI y Redis

Aprende a desacoplar la ejecución de tus agentes de IA de la respuesta de la API para crear sistemas que escalen horizontalmente.

Lo esencial en 3 líneas

1. **Qué construís:** una API con FastAPI que NO corre el agente dentro del request: lo encola en Redis y te devuelve un Job ID al instante.
2. **Por qué:** los flujos de agentes tardan 30-120s; mantener la conexión HTTP abierta se cae por timeout y satura el servidor.
3. **El patrón:** FastAPI (recibe y da el Job ID) → Redis (cola) → Worker (ejecuta el grafo) → el cliente consulta el estado por *polling*.

El problema de la latencia en IA

Cuando construimos sistemas multi-agente complejos, nos enfrentamos a un reto de ingeniería: **el tiempo de respuesta**. Un flujo de agentes que consulta herramientas, navega por internet y procesa datos en una base vectorial puede tardar 30, 60 o incluso 120 segundos.

Mantener una conexión HTTP abierta durante tanto tiempo es peligroso: los firewalls cortan la conexión por timeout, el usuario piensa que el sistema "se colgó" y tu servidor de API se queda sin recursos esperando a que el LLM termine.

La Solución: Arquitectura de Procesamiento Desacoplado

La técnica estándar en la industria para manejar tareas pesadas es el **procesamiento asíncrono con colas de mensajes**. En este esquema:

1. **FastAPI (El Recepcionista):** Recibe la petición, valida los datos, le da al usuario un "Ticket ID" (Job ID) y guarda la tarea en un buzón. Responde en milisegundos.
2. **Redis (El Buzón):** Actúa como *Message Broker*. Almacena las tareas pendientes de forma segura hasta que alguien las recoja.
3. **Worker (El Especialista):** Un proceso independiente (que puede escalar a múltiples instancias) que saca tareas de la cola, ejecuta el grafo de LangGraph y guarda el resultado final en Redis.

¿Por qué Redis?

Redis no es solo un caché. Gracias a sus estructuras de datos como **Lists** (para colas simples) o **Streams** (para arquitecturas de eventos), es el compañero ideal para FastAPI. Su velocidad en memoria permite coordinar miles de tareas con latencia casi nula.

Patrón: Polling (Sondeo)

En este nivel intermedio, implementaremos el patrón de **Polling**: el cliente pregunta periódicamente al endpoint `/status/{job_id}` si la tarea ya terminó. Es simple, robusto y fácil de implementar comparado con WebSockets en infraestructuras de nube complejas.

Errores Comunes

- **Ejecutar IA en el endpoint directamente:** Bloqueas el hilo de ejecución y tu API deja de responder a otros usuarios.

- **No persistir el estado inicial:** Si el worker es más rápido que la escritura del estado "pending", el cliente podría consultar el estado y recibir un error 404 porque el registro aún no existe. Siempre registra el estado *antes* de encolar la tarea.

Actividad evaluable · parte del proyecto integrador

API de producción y monitoreo activo

Instrucciones para la construcción de una API de producción que integra sistemas multi-agente con observabilidad y gestión de colas.

Qué construir

Debes entregar el código fuente de una **API REST (FastAPI)** que exponga tu sistema multi-agente desarrollado en el Módulo 6, con las siguientes capacidades nuevas:

1. **Endpoints Asíncronos:** La API no debe bloquearse. Debe recibir la tarea, encolarla y devolver un ID.
2. **Gestión de Estado con Redis:** Uso de Redis para persistir el estado de los trabajos y los checkpoints de LangGraph.
3. **Capa de Observabilidad:** Integración activa de instrumentación para enviar trazas de cada ejecución a una plataforma de monitoreo.
4. **Flujo Human-in-the-loop (HITL):** Implementación de una pausa obligatoria para aprobación humana en tareas que el sistema identifique como "críticas" (ej. ejecución de una herramienta con efectos secundarios o costo muy alto).

Pasos Sugeridos

1. **Refactorización del Grafo:** Toma tu orquestador multi-agente del Módulo 6 y asegúrate de que use un `RedisSaver` o similar para persistir el estado. Define un punto de interrupción en una arista o nodo crítico.
2. **Wrappers de Observabilidad:** Configura las variables de entorno para LangSmith o inicializa el colector de Arize Phoenix mediante OpenInference. Asegúrate de decorar tus funciones de llamada a LLM.
3. **Arquitectura de la API:** Crea la aplicación FastAPI. Implementa el patrón de "Worker" (puede ser un thread asíncrono simple en esta etapa o un proceso separado con Celery/Arq si buscas excelencia).
4. **Pruebas de Carga Sencillas:** Envía 5 peticiones concurrentes y verifica en tu dashboard de monitoreo cómo se comportan las trazas y la latencia.

Errores comunes a evitar

- **Bloquear el Event Loop:** Evita hacer llamadas `sync` a bases de datos o LLMs dentro de los endpoints de FastAPI sin usar `run_in_threadpool` o `async` nativo. Esto matará el rendimiento.
- **Ignorar el manejo de errores en Background Tasks:** Si el agente falla en segundo plano, el sistema debe capturar la excepción y actualizar el estado en Redis a `FAILED`, de lo contrario, el cliente se quedará esperando en un loop infinito de polling.

Qué entregás y en qué formato

- **Tipo:** Código — un repositorio de GitHub.
- **Artefacto concreto:** repo con la API FastAPI asíncrona, Redis para el estado, observabilidad (LangSmith o Arize Phoenix), el nodo Human-in-the-loop, el archivo de requerimientos, Docker Compose (opcional) y capturas del dashboard de trazas.
- **Qué NO hace falta:** no hay documento; las capturas del dashboard son evidencia de las trazas, no un informe.

Así se ve el repo esperado (estructura de referencia)

Tomalo de molde y adaptalo a tu implementación. Esta estructura cubre 1:1 lo que evalúa la rúbrica:

CÓDIGO

```
mi-api-agente/  
├─ app/  
│   ├── main.py          # FastAPI: POST /tasks → encola y devuelve job_id · GET /  
│   │                   tasks/{id} → estado (no bloquea)  
│   ├── graph.py         # tu orquestador multi-agente del M6 + checkpointer de  
│   │                   Redis (RedisSaver)  
│   ├── worker.py        # corre la tarea en segundo plano y actualiza estado en  
│   │                   Redis (PENDING/RUNNING/FAILED/DONE)  
│   ├── observability.py # init de Arize Phoenix o LangSmith (decoradores de  
│   │                   trazas)  
│   └─ hitl.py           # nodo de aprobación humana: interrupt + endpoint POST /  
│                   tasks/{id}/approve  
├─ requirements.txt  
├─ docker-compose.yml    # (opcional) app + redis  
├─ .env.example          # REDIS_URL y claves (sin secretos reales)  
├─ screenshots/          # capturas del dashboard de trazas (evidencia de  
│                   observabilidad)  
└─ README.md             # cómo levantar Redis + la API y cómo probar las 5  
peticiones concurrentes
```

Checklist de entrega (revisá antes de subir)

- Endpoint **asíncrono** que encola y devuelve un `job_id` sin bloquear el event loop. (→ Orquestación Asíncrona y API, 30%)
- Estado del job **persistido en Redis**, incluyendo el paso a `FAILED` ante una excepción. (→ Persistencia con Redis, 25%)
- **Trazas visibles** en el dashboard (Phoenix/LangSmith), con captura en `/screenshots`. (→ Observabilidad, 20%)
- Nodo **Human-in-the-loop** que pausa la ejecución hasta recibir aprobación externa. (→ HITL, 15%)
- **README** con los pasos para inicializar Redis y la API, y `requirements.txt`. (→ Documentación y estructura, 10%)
- Repo **público**: probá un `git clone` en limpio antes de entregar.

Pasos para la entrega

1. Crea una estructura de proyecto Python 3.12+.
2. Implementa tu sistema multi-agente del Módulo 6 dentro de un orquestador que use Redis como almacenamiento de estado.
3. Desarrolla la API con FastAPI utilizando los endpoints asíncronos mencionados en el briefing.
4. Configura Arize Phoenix o LangSmith para capturar trazas de cada nodo del agente.
5. Implementa un nodo de 'Aprobación Humana' que detenga la ejecución del grafo hasta recibir un input externo.
6. Sube el código a un repositorio y documenta en el README cómo inicializar Redis y la API.

Entregable

Formato: repositorio

Repositorio de GitHub con el código de la API, archivo de requerimientos, configuración de Docker Compose (opcional pero recomendado) y capturas de pantalla del dashboard de observabilidad con trazas activas.

Objetivos de aprendizaje

- Desplegar una API asíncrona utilizando FastAPI y Redis para gestionar tareas de IA de larga duración.
- Configurar sistemas de observabilidad (Arize Phoenix o LangSmith) para monitorear costos y latencia.
- Implementar flujos de interacción humana (Human-in-the-loop) dentro de grafos de agentes asíncronos.
- Diseñar mecanismos de persistencia de estado para orquestadores multi-agente complejos.

Rúbrica de evaluación

Evaluación de la capacidad para transicionar un sistema multi-agente a un entorno productivo, integrando persistencia de estado con Redis, endpoints asíncronos en FastAPI y observabilidad detallada.

Criterio	Descripción	Peso
Orquestación Asíncrona y API	Evaluación de la implementación de FastAPI y la gestión de endpoints asíncronos para el flujo de agentes.	30%
Persistencia de Estado con Redis	Capacidad para configurar e integrar Redis como almacenamiento de checkpoints para la persistencia de hilos de conversación.	25%
Observabilidad y Trazado	Implementación de herramientas de monitoreo (Arize Phoenix o LangSmith) para capturar el ciclo de vida de los mensajes.	20%
Intervención Humana (Human-in-the-loop)	Implementación de un nodo de aprobación que detenga el flujo hasta recibir una señal externa.	15%
Documentación y Estructura Profesional	Calidad del repositorio, README, archivo de requerimientos y despliegue inicial.	10%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 8

Capstone: entrega final

Consolidar todas las capas del sistema inteligente en un entregable final profesional y listo para producción.

Sistema Intelligence de grado de producción

Briefing detallado para la entrega final del programa, integrando todos los componentes del sistema de IA escalable.

Lo esencial en 3 líneas

1. **Qué entregás:** un repositorio de GitHub con un sistema de IA completo que integra RAG + agentes (con Supervisor) + API profesional + trazabilidad.
2. **De dónde sale:** ensamblás y refactorizás tus pre-entregas de los módulos anteriores (todo async, con Checkpointer de LangGraph y validación con Pydantic).
3. **Cómo se evalúa:** con la rúbrica de la entrega final (más abajo en esta unidad).

El Camino Recorrido

Para llegar aquí, has superado hitos técnicos fundamentales que ahora se ensamblan:

1. **Cimientos Asíncronos & Abstracción:** Implementaste clientes robustos que manejan streaming y latencia.
2. **RAG Avanzado:** Diseñaste estrategias de recuperación híbrida y gestión de metadatos para reducir alucinaciones.
3. **Inteligencia Colectiva:** Creaste una arquitectura de agentes donde un **Supervisor** delega tareas de forma autónoma.
4. **Robustez & Despliegue:** Preparaste el entorno para producción con observabilidad (Arize Phoenix) y escalabilidad (Redis/FastAPI).

Descripción del Entregable

Debes entregar un **repositorio de GitHub** que contenga un sistema de IA completo ("The Intelligence System"). Este sistema debe permitir a un usuario final (o proceso automatizado) resolver una tarea compleja que requiera:

- Acceso a conocimiento específico (vía RAG).
- Razonamiento multi-paso (vía Agentes).
- Interfaz de API profesional y documentada.
- Trazabilidad completa de lo que ocurre "bajo el capó".

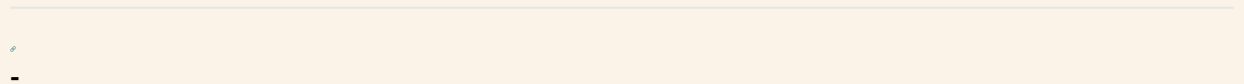
Errores Comunes a Evitar

- **Hard-coding:** Evita poner IDs de índices de Pinecone o modelos directamente en el código; usa variables de entorno.

- **Sincronicidad:** No mezcles llamadas síncronas (`requests`) dentro de funciones `async def` ; usa `httpx` o los SDKs asíncronos oficiales.
- **Falta de Trazas:** Entregar un sistema que "funciona" pero cuya ejecución es una caja negra (sin integración de logs o traces).
- **Alcance del despliegue:** este entregable se evalúa con **despliegue local** (Docker Compose o `run.sh`). **No se pide CI/CD** (GitHub Actions, pipelines de integración continua) ni infraestructura en la nube: no suma ni resta en la rúbrica.

Cierre del Programa

Este sistema es tu carta de presentación como **AI Architect**. Has construido una pieza de ingeniería capaz de razonar, recordar y escalar. Tras esta entrega, estarás listo para enfrentar retos de implementación de IA en entornos corporativos reales. **¡Es hora de consolidar tu visión! Estamos ansiosos por ver tu arquitectura en funcionamiento.**



¡Ya está disponible el material! Hacé clic en el enlace de abajo para repasar la presentación de la clase de hoy y refrescar los conceptos clave.

[Ver presentación aquí](#)

Pasos para la entrega

1. **Integración de Código:** Mueve la lógica de tus pre-entregas de los Módulos 6 y 7 a una estructura de proyecto unificada.
2. **Refactorización:** Asegurate de que toda la comunicación con LLMs y bases de datos sea asíncrona (`async/await`).
3. **Persistencia:** Implementa un `Checkpointner` en LangGraph para que las conversaciones no se pierdan al reiniciar el servidor.
4. **Validación:** Usa Pydantic para validar todas las entradas y salidas de la API y de las herramientas de los agentes.
5. **Observabilidad:** Realiza al menos 5 pruebas del sistema y asegurate de que las trazas se visualicen correctamente en Arize Phoenix o LangSmith.
6. **README:** Escribe la documentación incluyendo un diagrama de flujo del grafo de agentes.
7. **Despliegue Local:** Asegurate de que el proyecto pueda levantarse con una sola instrucción (ej. Docker Compose o un script `run.sh`).

Entregable

Formato: repositorio

Repositorio de GitHub con el código del sistema completo (Python 3.12+), archivos de configuración, documentación técnica y trazas de observabilidad configuradas.

Objetivos de aprendizaje

- El estudiante será capaz de consolidar múltiples componentes de un sistema de IA (RAG, Agentes, API) en un único repositorio profesional.
- El estudiante será capaz de aplicar principios de arquitectura limpia y desarrollo asíncrono para asegurar la producción del sistema.
- El estudiante será capaz de configurar y demostrar la observabilidad del sistema mediante trazas y monitoreo de costos.
- El estudiante será capaz de documentar decisiones técnicas de arquitectura para una audiencia de ingeniería senior.

Rúbrica de evaluación

Esta rúbrica evalúa la capacidad del estudiante para integrar componentes de IA, bases de datos vectoriales, orquestación de agentes y observabilidad en un sistema unificado, asíncrono y listo para producción, reflejando las competencias de un AI Architect.

Criterio	Descripción	Peso
Arquitectura y Coherencia del Sistema	Evalúa la integración de las capas de los módulos anteriores (LLMs, RAG, Agentes) en un flujo unificado y lógico.	30%
Robustez Técnica: Asincronía y Validación	Uso de patrones async/await en Python 3.12 y validación de datos con Pydantic.	25%
Persistencia y Observabilidad	Implementación de Checkpointers en LangGraph y trazado de ejecución con herramientas externas.	20%
Documentación y Despliegue	Claridad del README, diagrama de arquitectura y facilidad de ejecución (Docker/Scripts).	15%
Configuración de Entorno y Estándares de Código	Uso de Python 3.12+, gestión de dependencias y variables de entorno.	10%

Total: 100 pts | Aprobación: 70 pts

Material complementario

Palabras clave

- **Asyncio:** Es una librería de Python diseñada para escribir código que maneja múltiples tareas al mismo tiempo sin bloquear la ejecución del programa. En sistemas de IA, permite que tu aplicación

realice otras operaciones mientras espera la respuesta de un servidor externo de lenguaje, mejorando drásticamente la fluidez y velocidad del sistema.

- **RAG (Retrieval-Augmented Generation):** Es una arquitectura que combina la capacidad de respuesta de los modelos de IA con el acceso a datos externos específicos y actualizados. Funciona buscando información relevante en una base de datos propia antes de generar una respuesta, lo que reduce errores y asegura que la IA no invente datos inexistentes.
- **LCEL (LangChain Expression Language):** Es una sintaxis declarativa que facilita la unión de diferentes componentes de software como si fueran piezas de un rompecabezas para crear flujos de datos. Permite pasar información de un proceso a otro de forma automática, manejando errores y tareas paralelas de manera simplificada dentro del ecosistema de LangChain.
- **Vector Database (Base de Datos Vectorial):** Es un tipo especializado de base de datos que almacena información en forma de coordenadas numéricas llamadas vectores. Esto permite a la IA buscar datos basándose en el significado o concepto de las palabras en lugar de buscar coincidencias exactas, permitiendo encontrar documentos similares semánticamente.
- **LangGraph:** Es una herramienta diseñada para construir flujos de trabajo cíclicos donde la IA puede volver sobre sus propios pasos para corregir errores o analizar mejor un problema. A diferencia de las cadenas lineales, permite crear agentes que razonan en bucle hasta que consideran que han resuelto la tarea asignada.
- **Tool Calling (Llamado a Herramientas):** Es la capacidad que tiene un modelo de lenguaje para decidir por sí mismo cuándo necesita utilizar una función externa, como una calculadora o un buscador web. La IA genera los parámetros necesarios para ejecutar la acción y luego interpreta los resultados obtenidos para continuar con su respuesta.
- **Observabilidad:** Es el proceso de monitorear y rastrear cada paso interno que da un sistema de IA desde que recibe una petición hasta que entrega un resultado. Utilizando herramientas como LangSmith, los desarrolladores pueden ver dónde ocurrió un error, cuánto tiempo tardó cada proceso y cuánto dinero costó cada interacción.
- **Agentic Workflow (Flujo Agéntico):** Se refiere a un diseño de software donde la IA actúa como un agente con autonomía para tomar decisiones sobre qué pasos seguir a continuación. En lugar de seguir una ruta fija, el sistema evalúa la situación y elige dinámicamente qué herramientas o agentes auxiliares necesita para completar un objetivo.
- **Chunking:** Es la técnica de dividir documentos extensos en fragmentos más pequeños y manejables antes de procesarlos. Esto es vital para que la IA no se pierda en textos demasiado largos y pueda encontrar con precisión el párrafo exacto que contiene la respuesta a una pregunta específica.
- **Re-ranking:** Es una capa de optimización que revisa los resultados obtenidos de una búsqueda inicial y los vuelve a clasificar según su relevancia real para el usuario. Ayuda a filtrar ruidos y asegura que los mejores fragmentos de información sean los que finalmente se entreguen al modelo de IA para generar la respuesta.

Análisis profundo

El Arte del Manejo de Estados en Sistemas de IA Asíncronos

En el desarrollo de software tradicional, el estado suele ser lineal; sin embargo, en la arquitectura de un 'AI Architect', el manejo asíncrono del estado es el sistema circulatorio que permite la escalabilidad. ¿Por qué es crucial? Porque los LLMs son servicios externos con latencias impredecibles que pueden bloquear tu aplicación si no se gestionan con Python 3.12 y Asyncio. Si un desarrollador ignora el paradigma no bloqueante, el sistema colapsará bajo carga mínima, creando cuellos de botella donde los usuarios esperan segundos por una respuesta que podría haberse gestionado en paralelo.

Imagina que estás coordinando una cocina de alta gama. Si el chef principal (el hilo de ejecución) espera a que el agua hierva sin hacer nada más, la cocina es ineficiente. Asyncio permite que el chef prepare la salsa mientras el agua alcanza la temperatura. En el contexto de IA, esto significa que mientras esperamos los tokens de OpenAI, nuestro sistema puede estar consultando un metadato en ChromaDB o validando un esquema intermedio. Empresas como Uber o Netflix utilizan patrones de asincronía similares para manejar millones de peticiones concurrentes sin multiplicar linealmente sus recursos de hardware.

Un error común es confundir 'paralelismo' con 'asincronía'. Muchos estudiantes intentan usar threading para LLMs, lo cual es ineficiente debido al Global Interpreter Lock (GIL) de Python. La clave aquí es el Event Loop de la versión 3.12, que optimiza la conmutación de tareas I/O-bound. Al dominar esto, transformas un script de juguete en un motor de producción capaz de sostener flujos complejos de streaming de datos, reduciendo el 'Time to First Token' percibido por el usuario final, un KPI vital en la industria actual.

La Dualidad del RAG: De la Geometría Vectorial al Re-Ranking Semántico

Muchos desarrolladores creen que implementar un sistema Retrieval-Augmented Generation (RAG) consiste simplemente en subir PDFs a una base de datos vectorial. Esta simplificación es peligrosa: produce sistemas que sufren de 'alucinaciones informadas', donde el modelo recupera datos irrelevantes pero con alta similitud matemática. El problema real que resolvemos no es de almacenamiento, sino de relevancia semántica. ¿Cómo distinguimos un documento que 'se parece' de uno que realmente 'responde'?

El proceso comienza con la geometría del lenguaje: los embeddings. Estos transforman palabras en coordenadas en un espacio de n-dimensiones. Sin embargo, la similitud de coseno a veces falla porque el lenguaje es ambiguo. Aquí es donde entra el 'Re-ranking' y la recuperación híbrida. Mientras que herramientas como Pinecone gestionan la escala masiva de vectores, un re-ranker (como los modelos de Cohere) actúa como un filtro secundario de alta precisión que reordena los resultados basándose en el contexto real de la pregunta.

Considera una aplicación de soporte legal: un error en el chunking (fragmentación) de un contrato podría separar una cláusula de su excepción, llevando a la IA a dar un consejo erróneo. Los arquitectos senior no solo eligen una Vector DB; diseñan estrategias de 'Parent Document Retrieval' o 'Small-to-Big chunks' para asegurar que el contexto nunca se pierda. En el programa, entender esta dualidad es la

diferencia entre un sistema que 'lee' y uno que realmente 'comprende' la base documental de una empresa, evitando que el ruido nuble la señal crítica.

Grafos de Agentes: Superando la Rigidez de las Cadenas Secuenciales

En los módulos iniciales usamos LangChain para crear cadenas, pero en el mundo real, los problemas no son lineales. El concepto de 'Agentes con LangGraph' aborda el desafío de la autonomía y el razonamiento cíclico. El problema con las cadenas tradicionales es que si el paso 2 falla, toda la secuencia muere. Los grafos permiten ciclos: si un agente detecta que su propia respuesta es insuficiente o que la herramienta devolvió un error, puede volver atrás, corregir y reintentar.

Piensa en este sistema como una mesa redonda de especialistas. No es una línea de ensamblaje, sino un equipo dinámico. En LangGraph, definimos 'Nodos' (los cerebros o herramientas) y 'Aristas' con lógica condicional (el camino que decide el flujo). El patrón 'Supervisor' que exploramos es el equivalente a un Project Manager en software: un agente cuya única función es decidir a qué otro agente especialista (QA, Codificador, Investigador) delegar la siguiente tarea.

Una confusión habitual es pensar que más agentes siempre es mejor. La realidad es que la 'entropía de agentes' puede disparar los costos y la latencia. La ingeniería de sistemas de IA busca el equilibrio: usar estados compartidos bien definidos para que cada agente sepa exactamente qué hizo el anterior. Al implementar checkpoints y persistencia, permitimos que procesos largos (como una auditoría de código que dura 10 minutos) puedan pausarse y reanudarse sin perder el progreso, una característica fundamental para cualquier aplicación de nivel empresarial que aspire a la resiliencia.

Preguntas frecuentes

P: ¿Por qué usamos Asyncio y Python 3.12 desde el inicio si los scripts de IA suelen ser lineales?

R: Muchos desarrolladores asumen que la IA es una tarea puramente computacional que ocurre en su máquina, pero en realidad, las aplicaciones de producción pasan el 90% del tiempo esperando respuestas de red de APIs externas (como OpenAI o Anthropic). Usar `asyncio` nos permite manejar cientos de peticiones simultáneas sin bloquear el hilo principal. Python 3.12 introduce mejoras significativas en la gestión de tareas asíncronas y tipos estáticos que son vitales para la robustez del sistema. Imagina que tu sistema es un restaurante: sin asincronía, el camarero (tu código) se quedaría parado frente a la cocina esperando a que salga el plato, sin poder atender a nadie más. Con la arquitectura que aprenderás en el Módulo 1, el camarero toma el pedido y sigue atendiendo mesas mientras la comida se prepara, maximizando la eficiencia de tu infraestructura.

P: ¿Cuál es la diferencia real entre LangChain (Cadenas) y LangGraph (Agentes) y cuándo debo saltar de uno a otro?

R: Esta es una confusión común. LangChain (LCEL) está diseñado para flujos acíclicos y deterministas: Paso A -> Paso B -> Paso C. Es excelente para pipelines de extracción de datos o RAG simple. Sin embargo, los problemas complejos del mundo real requieren 'bucles' de razonamiento donde el sistema decide si la respuesta es suficiente o si debe reintentar una tarea. Ahí entra LangGraph. Muchos estudiantes intentan forzar LangChain para que se comporte como un agente usando lógica

compleja de 'if/else', pero eso se vuelve inmanejable. La regla de oro que aplicaremos en este programa es: si tu flujo de trabajo es una línea recta o un árbol de decisión predefinido, usa LCEL (Módulo 2); si tu aplicación requiere que la IA 'piense', use herramientas y corrija sus propios errores en un ciclo, utiliza LangGraph (Módulo 5).

P: ¿Por qué aprender ChromaDB local si luego usaremos Pinecone en la nube?

R: Muchos estudiantes creen que las bases de datos vectoriales son intercambiables y que empezar por una local es un paso innecesario, pero la realidad de la ingeniería es distinta. ChromaDB nos permite entender la 'geometría del lenguaje' y el manejo de archivos localmente sin latencia de red ni costos de infraestructura, lo cual es ideal para la etapa de prototipado y unit testing. Una vez que entiendes las operaciones CRUD y el chunking (Módulo 3), escalar a Pinecone (Módulo 4) se vuelve un reto de arquitectura y no de conceptos básicos. En la industria, a menudo desarrollarás y testearás localmente para ahorrar costos, y solo desplegarás en soluciones cloud como Pinecone cuando necesites escalabilidad horizontal, alta disponibilidad y gestión de millones de vectores en producción.

P: ¿Es realmente necesario implementar 'Human-in-the-loop' en sistemas autónomos?

R: Existe la idea errónea de que un sistema de IA es 'mejor' cuanto más autonomía tenga. Sin embargo, en entornos corporativos de alta responsabilidad (finanzas, salud, legal), un sistema 100% autónomo es un riesgo inaceptable. El concepto de Human-in-the-loop (HITL) que vemos en el Módulo 7 no es un paso atrás, sino una característica de seguridad avanzada. Aprenderás a diseñar puntos de interrupción donde el agente 'levanta la mano' para pedir validación humana ante una duda o una acción crítica. Piensa en ello como el sistema de piloto automático de un avión: es extremadamente capaz, pero está diseñado para que el piloto humano supervise y tome el control en momentos clave. Esto es lo que diferencia a un proyecto escolar de una arquitectura de grado profesional.

P: ¿Cómo puedo evaluar si mis optimizaciones de RAG están funcionando realmente?

R: Muchos desarrolladores se frustran porque ajustan sus prompts o el tamaño de sus chunks 'por intuición', viendo si la respuesta les 'parece buena'. En este programa, eliminamos la subjetividad mediante métricas de evaluación automática y observabilidad (Módulo 4 y 7). Aprenderás a medir el 'Recall' (¿encontró el sistema la información relevante?) y la 'Precisión' (¿lo que encontró era realmente necesario?). Utilizaremos herramientas como Arize Phoenix y LangSmith para trazar cada paso de la cadena. Si no puedes medirlo, no puedes optimizarlo. El enfoque de AI Architect consiste en tratar la evaluación de la IA como si fueran tests unitarios en el software tradicional: si cambias un parámetro, debes correr un set de evaluación para confirmar que el sistema no ha empezado a alucinar.

P: ¿Qué tan avanzado debe ser mi nivel de Python para no perderme en el curso?

R: El nivel solicitado es INTERMEDIO. Esto no significa que debas ser un experto en algoritmos complejos, sino que debes sentirte cómodo con la programación orientada a objetos (clases y herencia), decoradores y tipado básico. Lo más importante es que estés dispuesto a adoptar el paradigma funcional de LangChain y la lógica asíncrona de Python 3.12. Muchos desarrolladores seniors de otros lenguajes se frustran con la IA porque intentan escribir Python como si fuera Java o

C#. Aquí aprenderás a escribir Python 'idiomático' para IA, aprovechando las librerías modernas. Si conoces cómo estructurar un proyecto con módulos y entiendes qué es una promesa o un 'future', tienes la base necesaria para escalar con nosotros hacia arquitecturas complejas.

Conclusiones

De Scripts Lineales a Arquitecturas de Agentes Autónomos

Has trascendido la fase de prototipado básico para dominar el diseño de sistemas inteligentes circulares y modulares. La transición desde el Módulo 1 (abstracción base) hasta el Módulo 6 (multi-agentes) representa un cambio de paradigma profesional: dejas de programar flujos de texto rígidos para diseñar ecosistemas que razonan y colaboran. Este enfoque te permite abordar problemas de negocio complejos donde una sola consulta no es suficiente, estructurando soluciones a través de supervisores que delegan tareas a especialistas. Ahora puedes gestionar la incertidumbre de la IA mediante estados persistentes en LangGraph, permitiendo que tus sistemas retomen procesos complejos exactamente donde los dejaron, una habilidad crítica para aplicaciones empresariales de larga duración.

La Dualidad RAG: Escalabilidad del Conocimiento y Verdad Técnica

A través de la integración de los Módulos 3 y 4, has construido un puente entre la lógica del LLM y los datos privados de una organización. La progresión de una base vectorial local (ChromaDB) a una infraestructura en la nube (Pinecone) no es solo un cambio de herramienta, sino una lección de escalabilidad horizontal. Has aprendido que un RAG profesional no se trata solo de 'conectar datos', sino de optimizar la recuperación mediante re-ranking e ingesta con metadatos avanzados para minimizar alucinaciones. En el mundo profesional, esta competencia te permite transformar silos de información estática en activos dinámicos que alimentan la generación de contenido con precisión quirúrgica y fundamentos sólidos.

Ingeniería de Producción: Observabilidad y el Factor Humano

El conocimiento técnico es incompleto sin la robustez operativa explorada en los Módulos 7 y 8. Has integrado una mentalidad de 'AI-Ops' que garantiza que tus sistemas no sean cajas negras. Al dominar herramientas como LangSmith y Arize Phoenix, ahora posees la capacidad de rastrear cada token, evaluar costos en tiempo real y detectar fallos de lógica antes de que afecten al usuario final. La incorporación de patrones 'Human-in-the-loop' asegura que, para decisiones críticas, tu arquitectura mantenga un equilibrio entre autonomía y supervisión humana. Este es el diferenciador que te permite presentar proyectos a niveles directivos con métricas claras de latencia, seguridad y retorno de inversión.

Capacidad de Transformación Profesional: El Arquitecto de IA de 2026

Al finalizar este programa, tu perfil ha evolucionado de un desarrollador de software tradicional a un Arquitecto de IA capaz de orquestar infraestructuras complejas. Lo que antes era un desafío de integración asíncrona y gestión de prompts, ahora es un flujo de trabajo optimizado que utiliza Python 3.12 y Redis para servir miles de peticiones simultáneas. Estás capacitado para liderar equipos técnicos

en la creación de productos que van desde sistemas de soporte autónomos hasta analistas de datos multi-agente. Tu valor en el mercado actual radica en la síntesis de tres pilares que has dominado: orquestación lógica, gestión persistente de la memoria y despliegue escalable en entornos de producción real.